VŨ ĐÌNH MINH DŨNG MSSV: N21DCCN113 CHUYÊN NGÀNH: TRÍ TUỆ NHÂN TẠO 2021-2026 Lớp:E21CQCNTT01-N
KIỀU TUẤN TRUNG ANH MSSV: N21DCCN004

TP. HCM

2025

**BỘ KHOA HỌC CÔNG NGHỆ**

**HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG**

---

# ĐỒ ÁN

# TỐT NGHIỆP ĐẠI HỌC

## Đề tài: "Xây dựng ứng dụng tái tạo tái tạo vật thể 3D từ ảnh 2D sử dụng kỹ thuật học sâu"


**Giảng viên hướng dẫn : Nguyễn Thị Tuyết Hải**

**Sinh viên thực hiện : Kiều Tuấn Trung Anh**
**Vũ Đình Minh Dũng**

**Mã số sinh viên : N21DCCN004**
**N21DCCN113**

**Lớp : E21CQCNTT01-N**

**Hệ : Đại học chính quy chất lượng cao**


**TP. HỒ CHÍ MINH, NĂM 2025**

**BỘ KHOA HỌC CÔNG NGHỆ**

**HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG**

---

# ĐỒ ÁN

# TỐT NGHIỆP ĐẠI HỌC

## Đề tài: "Xây dựng ứng dụng tái tạo tái tạo vật thể 3D từ ảnh 2D sử dụng kỹ thuật học sâu"

**Giảng viên hướng dẫn** : **Nguyễn Thị Tuyết Hải**

**Sinh viên thực hiện** : **Kiều Tuấn Trung Anh**
**Vũ Đình Minh Dũng**

**Mã số sinh viên** : **N21DCCN004**
**N21DCCN113**

**Lớp** : **E21CQCNTT01-N**

**Hệ** : **Đại học chính quy chất lượng cao**

**TP. HỒ CHÍ MINH, NĂM 2025**

# ACKNOWLEDGEMENTS

We would like to express our deepest gratitude to our supervisor, Ms. Nguyen Thi Tuyet Hai, for her dedicated guidance, valuable insights, and continuous encouragement throughout the course of this graduation project. Her expertise and feedback were instrumental in helping us navigate the complexities of 3D scene reconstruction and SLAM technologies.

We also wish to thank the lecturers at the Posts and Telecommunications Institute of Technology (PTIT) for equipping us with the fundamental knowledge and providing an excellent academic environment during our years of study.

Finally, we are grateful to our families and friends for their unwavering support, patience, and understanding, which motivated us to overcome the challenges encountered during this research.

# TABLE OF CONTENTS

# SYMBOLS AND ACRONYMS

| Symbols and Acronyms | Meaning |
|---|---|
| SLAM | Simultaneous Localization and Mapping |
| RGB-D | RGB-Depth Camera |
| SOTA | State Of The Art |
| UAVs | Unmanned Aerial Vehicles |
| VR | Virtual Reality |
| AR | Augmented Reality |
| IMU | Inertial Measurement Unit |
| SfM | Structure from Motion |
| SfS | Shape from Shading |
| MDE | Monocular Depth Estimation |
| ViT | Vision Transformer |
| DPT | Dense Prediction Transformer |
| SSL | Self-Supervised Learning |
| VO | Visual Odometry |
| ORB | Oriented FAST and Rotated BRIEF |
| FAST | Features from Accelerated Segment Test |
| BRIEF | Binary Robust Independent Elementary Feature |
| BA | Bundle Adjustment |
| RANSAC | Random Sample Consensus |
| PnP | Perspective-n-Point |
| ATE | Absolute Trajectory Error |
| RPE | Relative Pose Error |

*Table 1: List of Symbols and Acronyms*

# LIST OF FIGURES

# LIST OF TABLES

# INTRODUCTION

## 1. Background and Motivation

When computer vision first began, people dreamed that computers would one day see and understand the world like humans do. The idea of letting machines explore unknown places felt exciting and almost magical, inspiring many researchers to work tirelessly on it. At first, we thought it wouldn't be too hard, but the journey turned out to be much tougher.

In a computer, things like flowers, trees, insects, birds, and animals are not seen as we see them. Instead, they appear as large grids of numbers. Teaching a computer to understand an image is as hard as asking a person to make sense of random number blocks. We didn't even fully understand how humans make sense of images, let alone how to teach a machine to do the same.

But after many years of effort, progress finally appeared. Thanks to Artificial Intelligence and Machine Learning, computers can now recognize objects, faces, voices, and text—although they do it using probability and statistics, which is still very different from how humans think.

## 2. Problem Statement:

This report is about **SLAM**. So what is **SLAM** ? It stands for **Simultaneous Localization and Mapping**. It usually refers to a robot or a moving rigid body, equipped with a specific sensor, estimates its **motion** and builds a **model** of the surrounding environment, without any prior information. In other words, it is a problem of how to **estimate the location of a sensor** itself, while estimating the model of the environment.

The first problem in SLAM is used its sensor information to approximate the depth of objects it seen, then references could be picked for localizing itself in the world. If the sensor referred is a camera, it is called Visual SLAM. The camera type can be monocular or stereo or RGB-D (ToF – Time Of Flight), this work will mainly refered about RGB-D type which basically a monocular camera equipping with an IR (Infrared) sensor to output RGB image along with corresponding depth field image whose pixels can give distance of objects in the scene. But out of curiosity we are going to attempt to **replace RGB-D with a monocular camera pipelining with a SOTA depth estimation model** and see how well this system perform under various enviroments.

Knowing the camera location could be very useful in many areas: indoor sweeping machines and mobile robots, self-driving cars, UAVs, VR and AR. Without it, the sweeping machine cannot maneuver in a room autonomously but wandering blindly instead; domestic robots can not follow in structions to accurately reach a specific room; Virtual reality devices will always be limited within a seat.

SLAM technology has undergone a significant change and breakthrough in both theory and practice and is gradually moving from laboratories into the real-world. It has over 30 years of research history, and it has been a hot topic in both robotics and computer vision communities. Although SLAM's theoretical framework has basically become mature, implementing a complete system is still very challenging and requires a high level of technical expertise.

This report will layout as follow:

In CHAPTER 1, we first introduced the theoretical background knowledge that must require for the following chapters.

In CHAPTER 2, after taking a glimsp of key knowledge that enabling SLAM to work, we went into detail of a specific implementation of SLAM called **ORB-SLAM** and our work included a modify version of this system which will also be discussed.

The last CHAPTER 3, we showed the evaluation performance of our modify system against the original.

The APPENDIX explained about some implement detail of ORB-SLAM system.

# TASK ALLOCATION TABLE

# CHAPTER 1: THEORETICAL BACKGROUND

## 1.1 Computer Vision Fundamentals

### 1.1.1 The Pinhole Camera Model

The Pinhole Camera Model is the simplest and most widely used geometric model in computer vision. It describes the mathematical relationship between the coordinates of a 3D point in space and its projection onto the 2D image plane.

In this model, light rays from an object pass through a single point called the Camera Center (or Optical Center) and project an inverted image onto the Image Plane. For mathematical convenience, the image plane is often conceptualized as being placed in front of the camera center to maintain the upright orientation of the image.

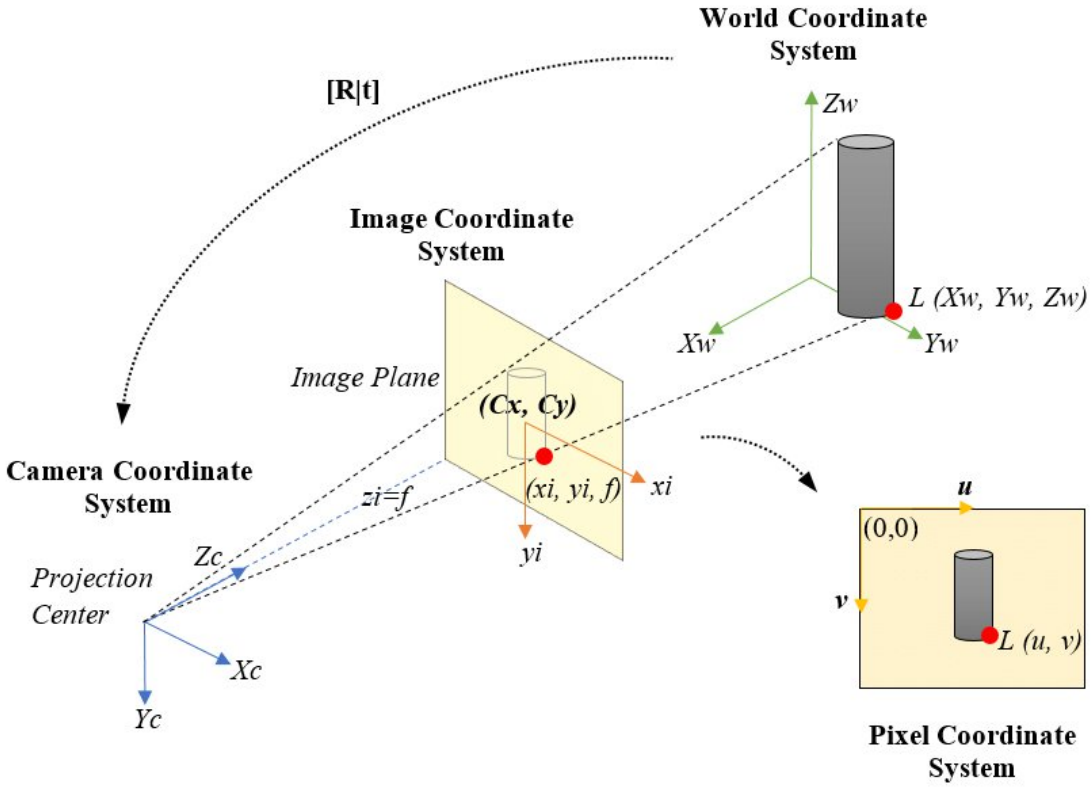


*Figure 1.1: Diagram of the pinhole camera model showing 3D to 2D projection geometry with an inverted image. Source: ResearchGate.*

Mathematically, if a point in the 3D camera coordinate system is $P(X, Y, Z)$, its projection $p(x, y)$ on the image plane is derived using the properties of similar triangles:

$$x = f \cdot \frac{X}{Z}, \quad y = f \cdot \frac{Y}{Z}$$

Where, $f$ is the focal length (distance between the Camera Center and the Image Plane), $Z$ is the depth of the object along the optical axis.

### 1.1.2 Coordinate Systems

To reconstruct a 3D scene, we must define the transformations between three distinct coordinate systems:

- **World Coordinate System** $(X_w, Y_w, Z_w)$: An absolute reference frame for the object in the real world (e.g., a corner of the room).
- **Camera Coordinate System** $(X_c, Y_c, Z_c)$**:** Attached to the camera device (the mobile phone). The origin is the lens center, and the $Z_c$-axis points forward along the optical axis.
- **Image Coordinate System** $(X_i, Y_i)$: The 2D coordinate system of the digital image, usually measured in pixels with the origin (0, 0) at the top-left corner.

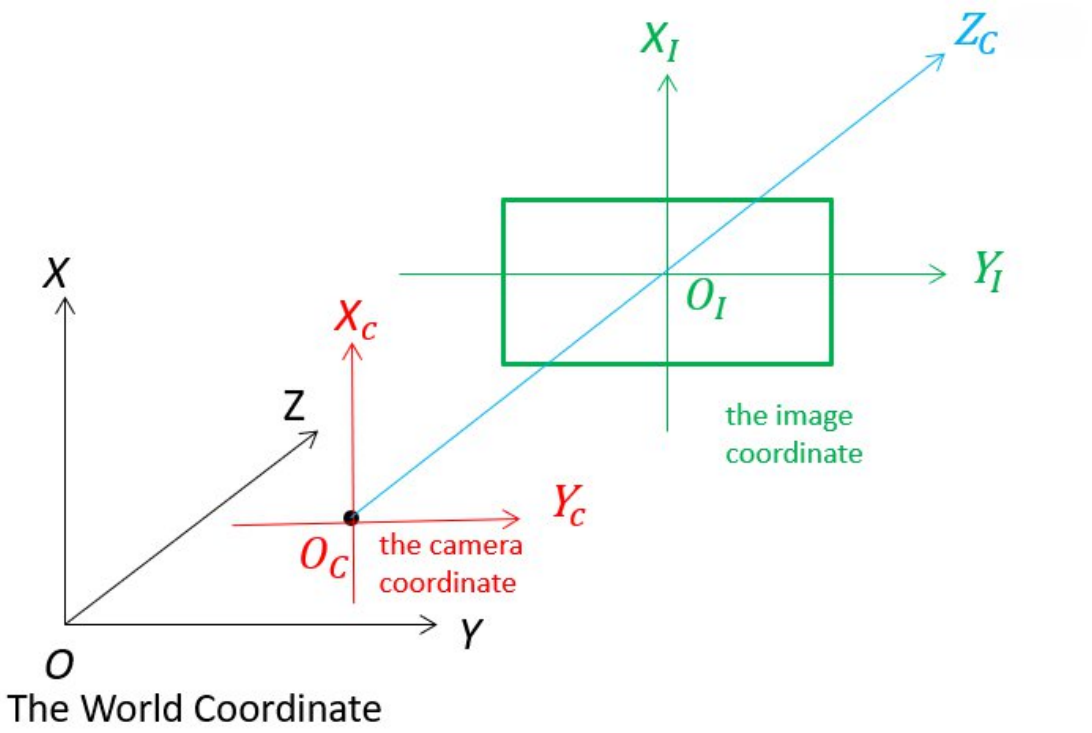


*Figure 1.2: Visualization of World, Camera, and Image Coordinate Systems.*

*Source: ResearchGate.*

The transformation from World Coordinates to Pixel Coordinates involves two steps:

1) **Extrinsic Transformation (World → Camera):** Uses a rotation matrix *R* and a translation vector *t* (handled by the SLAM module in this project).
2) **Intrinsic Transformation (Camera → Image):** Uses the internal properties of the camera (handled by the Calibration module).

### 1.1.3 Camera Intrinsic Parameters

The intrinsic parameters link the normalized camera coordinates to the actual pixel coordinates in the digital image. This transformation accounts for the focal length and the optical center offset. The mapping is represented by the Intrinsic Matrix *K*:

$$K = \begin{bmatrix} f_x & s & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}$$

#### 1.1.3.1 Focal Length Parameters

The focal length parameters $f_x$ and $f_y$ represent the effective focal lengths expressed in pixel units along the $x$ and $y$ axes respectively. These parameters relate the physical focal length $f$ (measured in millimeters) to the pixel dimensions through the sensor's pixel size:

$$f_x = \frac{f}{pixel\_size_x}, \; f_y = \frac{f}{pixel\_size_y}$$

In most modern cameras, pixels are square, resulting in $f_x = f_y$. However, for cameras with rectangular pixels, these values differ.

#### 1.1.3.2 Principal Point

The principal point $(c_x, c_y)$ represents the intersection of the optical axis with the image plane, expressed in pixel coordinates. Ideally, this point should be located at the image center, but manufacturing imperfections can cause it to deviate:

$$c_x \approx \frac{image\_width}{2}, \; c_y \approx \frac{image\_height}{2}$$

#### 1.1.3.3 Skew Coefficient

The skew coefficient s accounts for any non-perpendicularity between the sensor's $x$ and $y$ axes. In modern sensors, this parameter is typically zero since the pixel arrays are manufactured with perpendicular axes:

$$s = f_x \cot(\theta)$$

Where $\theta$ is the angle between the pixel axes, ideally 90°.

### 1.1.4 Projection Transformation

The intrinsic matrix transforms a 3D point $\boldsymbol{P_c} = [X_c, Y_c, Z_c]^T$ in camera coordinates to homogeneous image coordinates $x = [\tilde{x}, \tilde{y}, \tilde{w}]^T$:

$$x = K\boldsymbol{P_c} = \begin{bmatrix} f_x X_c + s Y_c + c_x Z_c \\ f_y Y_c + c_y Z_c \\ Z_c \end{bmatrix}$$

Converting to Cartesian coordinates by dividing by the third component:

$$x = \frac{f_x X_c + s Y_c + c_x Z_c}{Z_c},\ y = \frac{f_y Y_c + c_y Z_c}{Z_c}$$

This simplifies to the standard perspective projection equations when $s = 0$:

$$x = f_x \frac{X_c}{Z_c} + c_x,\ y = f_y \frac{Y_c}{Z_c} + c_y$$

### 1.1.5 Matrix Decomposition

The intrinsic matrix can be decomposed into a sequence of elementary transformations:

$$K = \begin{bmatrix} 1 & 0 & c_x \\ 0 & 1 & c_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} f_x & s & 0 \\ 0 & f_y & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

This decomposition separates the scaling and shear effects from the translation of the principal point, providing intuitive understanding of each parameter's geometric role.

## 1.2 Lens Distortion

### 1.2.1 Overview and Physical Causes

Lens distortion represents deviations from the ideal pinhole camera model due to imperfections in the optical system. Real camera lenses introduce systematic geometric distortions that cause straight lines in the world to appear curved in images. These distortions arise from several physical factors including lens shape irregularities, manufacturing tolerances, and assembly misalignments[1].

*Figure 1.3: Comparison of an image with barrel lens distortion*

*and its corrected version demonstrating the effect of lens distortion correction methods*

### 1.2.2 Types of Distortion

#### 1.2.2.1 Radial Distortion

Radial distortion is the most significant type, caused by unequal bending of light rays at different radial distances from the optical center. Light rays near the lens edges bend more than those near the center, creating characteristic distortion patterns.

Two primary forms of radial distortion exist:

- Barrel Distortion: Negative radial displacement where straight lines bow outward from the image center, resembling the sides of a barrel
- Pincushion Distortion: Positive radial displacement where straight lines curve inward toward the center, creating a pincushion effect

The mathematical model for radial distortion uses a polynomial expansion in terms of the radial distance $r$ from the image center:

$$x_{distorted} = x_{undistorted}\left(1 + k_1 r^2 + k_2 r^4 + k_3 r^6 + \ldots\right)$$

$$y_{distorted} = y_{undistorted}\left(1 + k_1 r^2 + k_2 r^4 + k_3 r^6 + \ldots\right)$$

Where $r^2 = x_{distorted}^2 + y_{distorted}^2$ and $k_1, k_2, k_3 \ldots$ are the radial distortion coefficients.

#### 1.2.2.2 Tangential Distortion

Tangential distortion, also known as decentering distortion, occurs when the lens elements are not perfectly aligned with the image sensor plane during camera assembly. This misalignment causes points to shift in the tangential direction relative to the radial vector from the image center.

The tangential distortion model is expressed as:

$$\delta x = 2p_1 xy + p_2(r^2 + 2x^2)$$

$$\delta y = p_1(r^2 + 2y^2) + 2p_2 xy$$

where $p_1$ and $p_2$ are the tangential distortion coefficients, and $x$,$y$ are the normalized image coordinates.

### 1.2.3 Distortion Correction Process

The distortion correction process involves estimating the distortion parameters through camera calibration and then applying the inverse transformation to rectify images. The correction typically follows these steps:

- Parameter Estimation: Using calibration images with known geometric patterns to estimate distortion coefficients
- Coordinate Transformation: Converting pixel coordinates to normalized coordinates relative to the principal point
- Distortion Application: Applying the distortion model to map distorted coordinates to undistorted positions

- Image Rectification: Interpolating pixel values at corrected coordinate locations

The mathematical relationship for undistortion can be expressed as finding the undistorted coordinates $(x_u,y_u)$ given distorted coordinates $(x_d,y_d)$:

$$x_u = x_d + (x_d - x_c)(k_1 r^2 + k_2 r^4 + \ldots) + [p_1(r^2 + 2(x_d - x_c)^2) + 2p_2(x_d - x_c)(y_d - y_c)]$$

$$y_u = y_d + (y_d - y_c)(k_1 r^2 + k_2 r^4 + \ldots) + [2p_1(x_d - x_c)(y_d - y_c) + p_2(r^2 + 2(y_d - y_c)^2)]$$

where $(x_c,y_c)$ represents the distortion center, typically coincident with the principal point.

### 1.2.4 Combined Distortion Model

The complete lens distortion model combines both radial and tangential components, known as the Brown-Conrady model:

$$x_{corrected} = x_{distorted}\left(1 + k_1 r^2 + k_2 r^4 + k_3 r^6 + \ldots\right) + [2p_1 x_{distorted} y_{distorted} + p_2(r^2 + 2x_{distorted}^2)]$$

$$y_{corrected} = y_{distorted}(1 + k_1 r^2 + k_2 r^4 + k_3 r^6 + \ldots) + [p_1(r^2 + 2y_{distorted}^2) + 2p_2 x_{distorted} y_{distorted}]$$

where $x_{distorted}$ and $y_{distorted}$ are the distorted image coordinates, and $x_{corrected}$ and $y_{corrected}$ are the corrected image coordinates. The parameters $k_1$, $k_2$, $k_3$, $p_1$, and $p_2$ are the distortion coefficients that need to be estimated during the calibration process.

## 1.3 Monocular Depth Estimation

### 1.3.1 Overview of Monocular Depth Estimation

Monocular Depth Estimation is a fundamental computer vision task focused on predicting dense depth maps from a single RGB image. Fundamentally, this process involves inferring the distance of scene objects relative to a singular camera viewpoint, effectively recovering the third dimension from a two-dimensional input.

This technology is pivotal across various domains, including 3D scene reconstruction, AR, autonomous driving, and robotics. However, Monocular Depth Estimation remains a technically challenging problem often described as "ill-posed" because an infinite number of 3D scenes can project onto the same 2D image. Consequently, models must learn to interpret complex contextual cues and object relationships while accounting for environmental variables such as varying illumination, occlusion, and texture homogeneity.

There are two main depth estimation categories:

- **Absolute (Metric) Depth Estimation:** This approach focuses on recovering exact physical distance measurements from the camera to objects in the scene. Often referred to as metric depth estimation, this paradigm generates depth maps where pixel values correspond to precise real-world units (e.g., meters or feet). It is essential for applications requiring high spatial accuracy, such as navigation and obstacle avoidance.
- **Relative Depth Estimation:** Unlike the metric approach, relative depth estimation prioritizes the ordinal relationship between objects within a scene rather than their physical distance. The objective is to predict the depth ordering determining which objects are closer or further relative to one another without assigning specific numerical distance values.

### 1.3.2 Traditional vs. Deep Learning Approaches

#### 1.3.2.1 Traditional Method

##### 1.3.2.1.1 Structure from Motion (SfM):

Structure from Motion is the process of estimating the 3-D structure of a scene from a set of 2-D images. SfM is used in many applications, such as 3-D scanning , augmented reality, and visual simultaneous localization and mapping (vSLAM).

SfM can be computed in many different ways. The way in which you approach the problem depends on different factors, such as the number and type of cameras used, and whether the images are ordered. If the images are taken with a single calibrated camera, then the 3-D structure and camera motion can only be recovered up to scale. up to scale means that you can rescale the structure and the magnitude of the camera motion and still maintain observations. For example, if you put a camera close to an object, you can see the same image as when you enlarge the object and move the camera far away.

Despite its mathematical rigor, SfM faces several significant challenges in real-world deployment:

- **Scale Ambiguity:** As described, it cannot recover absolute metric depth (e.g., meters) without external references or sensors (like IMU), making it difficult for applications requiring precise measurements.
- **Texture Dependency:** SfM relies on matching "feature points" (corners, edges) across images. It often fails in **texture-less regions** (e.g., plain white walls, clear skies) because there are no features to track.
- **Sparsity:** The output is typically a **sparse point cloud** representing only the tracked features, rather than a dense depth map for every pixel.
- **Static Assumption:** It assumes the scene is static. Moving objects (people, cars) violate the geometric constraints and cause reconstruction errors.

#### 1.3.2.1.2 Shape from Shading (SfS):

Shape from Shading is a technique that reconstructs the 3-D shape of a surface by analyzing the variations in shading (brightness) within a single image. The underlying principle is that the brightness of a pixel depends on the orientation of the surface relative to the light source. By solving the irradiance equation, SfS estimates the surface normals and integrates them to recover depth.

While SfS can operate on a single image, it is rarely used as a standalone solution for general scene reconstruction due to:

- **Lighting Assumptions:** It typically assumes a known, single point light source and Lambertian surface reflectance (matte surfaces). Real-world lighting (multiple lights, ambient light) breaks these assumptions.
- **Albedo Ambiguity:** It is difficult to distinguish whether a change in pixel intensity is due to the surface geometry (slope) or the surface color (albedo pattern).
- **Optimization Complexity:** The mathematical optimization is computationally expensive and prone to getting stuck in local minima.

### 1.3.2.2 Modern Deep Learning Approaches

With the advent of Convolutional Neural Networks (CNNs) and Transformers, MDE has shifted towards a learning-based paradigm. Instead of relying on explicit geometric calculation, these models learn implicit depth cues (such as object size, occlusion, and perspective) from vast amounts of data.

**Supervised Learning Paradigm:**

In this approach, networks are trained using pairs of RGB images and corresponding ground-truth depth maps (captured by LiDAR or Kinect).

- **Pixel-wise Regression:** The network treats depth estimation as a regression problem, predicting a continuous depth value for every pixel.
- **Scale-Invariant Loss:** To handle scale ambiguity, Eigen et al. introduced the Scale-Invariant Log Loss, which penalizes relative depth errors rather than absolute differences:

$$\mathrm{L}(\mathrm{y}, \mathrm{y}^{*}) = \frac{1}{\mathrm{n}} \sum_{\mathrm{i}} d_i^2 - \frac{\lambda}{\mathrm{n}^2} \left( \sum_{\mathrm{i}} d_i \right)^2$$

Where $d_i = \log(y_i) - \log(\mathrm{y}_1^*)$ represents the difference between prediction and ground truth in log space.

**Network Architectures: From CNN to Transformers**

- **CNN-based (e.g., U-Net):** Early models used encoder-decoder architectures with Convolutional layers. While effective at capturing local details, they struggle with global context due to limited receptive fields.
- **Transformer-based (e.g., DINOv2, Depth Anything):** Modern approaches utilize **Vision Transformers (ViT)**. The *Self-Attention* mechanism allows the model to capture long-range dependencies across the entire image. This enables the model to infer depth in texture-less regions by looking at the global context (e.g., understanding the layout of a room from its corners).

**Self-Supervised Learning (SSL)**

To overcome the scarcity of labeled depth data, recent trends focus on SSL. The model is trained on monocular video sequences by learning to synthesize a target frame from source frames using predicted depth and pose. The objective is to minimize the photometric reconstruction error:

$$L_{photo} = \sum_{p} |I_t(p) - I_{s \to t}(p)|$$

## 1.4 The Depth Anything V2 Architecture

In this project, the Depth Anything V2 model is employed as the core depth estimation engine. Based on the research by Yang et al. (2024), Depth Anything V2 is a foundation model designed to overcome the limitations of labeled real-world datasets, offering superior fine-grained details and robustness compared to its predecessors (MiDaS, Depth Anything V1).

### 1.4.1 Motivation: The "Real Data" Dilemma

Conventional Monocular Depth Estimation (MDE) models rely heavily on large-scale real-world datasets labeled by depth sensors (e.g., LiDAR, Kinect). However, the Depth Anything V2 paper identifies a critical issue: Label Noise.

Real-world depth labels are often sparse, noisy, and misaligned. Training on such data causes the model to learn these imperfections, resulting in thickened edges and "flying pixels" (noise floating around objects). To address this, Depth Anything V2 introduces a paradigm shift: replacing real labeled data with high-quality synthetic data for training the teacher model.

### 1.4.2 Model Architecture

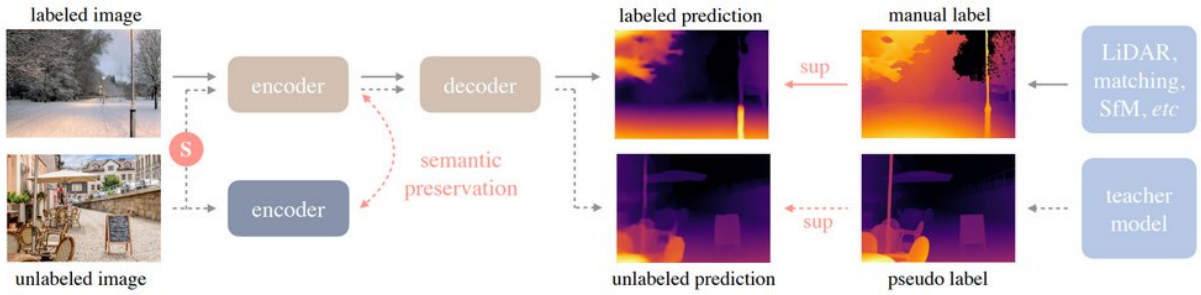


*Figure 1.4: Depth Anything V2 overview. Source: Roboflow*

Depth Anything V2 architecture combines a powerful transformer-based encoder (DINOv2), a decoder built on DPT, and a multi-stage training process, including synthetic pre-training, large-scale pseudo-label distillation, and optional metric fine-tuning, to deliver robust performance across both synthetic and real-world imagery. Following are the main components:

- **Backbone (Encoder):** The model utilizes DINOv2, a Vision Transformer (ViT) pre-trained with self-supervised learning. DINOv2 provides rich semantic representations, allowing the model to understand object categories and context, which is essential for inferring depth in complex scenes.
- **Depth Decoder:** A DPT (Dense Prediction Transformer) decoder is used on top of the encoder. It aggregates multi-scale features from different stages of the ViT to reconstruct a high-resolution depth map.
- **Loss Functions**: The architecture uses following loss function on training-time only: *Scale-and-Shift-Invariant Loss* ($L_{ssi}$), it is used to ensures depth consistency regardless of global scale or offset; *Gradient Matching Loss* ($L_{gm}$), it is used to sharpen edges and preserves object boundaries accurately. These two losses are combined with a typical weight ratio of:

$$L_{ssi}:L_{gm} = 1:2$$

### 1.4.3 The Three-Stage Training Pipeline

The superior performance of Depth Anything V2 stems from its unique three-stage distillation pipeline, designed to bridge the gap between synthetic precision and real-world diversity.

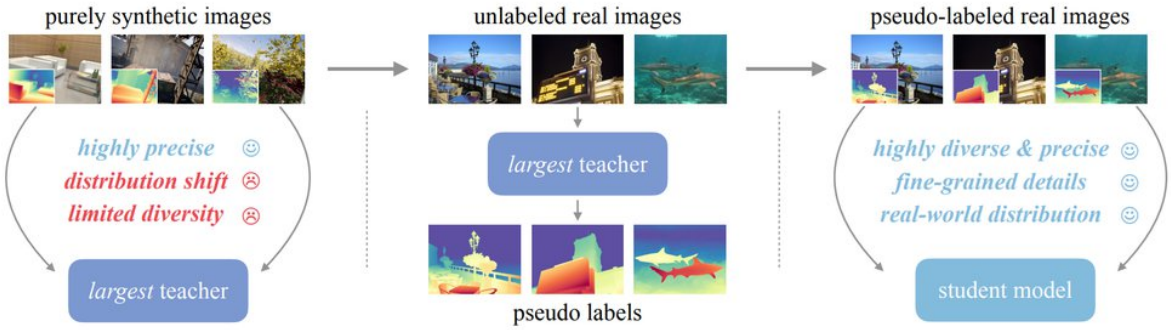


*Figure 1.5: Training Framework: A teacher model is trained on precise synthetic data, used to pseudo-label diverse real images, and student models are then trained on these high-quality pseudo labels for robust depth estimation. Source: Roboflow*

**Stage 1 Teacher Pre-training**: It train a large teacher model (e.g., DINOv2-G/Vit-Giant) on pure synthetic depth data (595K images). The purpose is to teach robust depth cues and fine-grained details from perfect ground truth.

**Stage 2 Pseudo-label Generation**: This stage use the trained teacher on unlabeled real images (~62 million). It automatically generate pseudo-ground-truth depth maps to mitigate synthetic-to-real domain shift.

**Stage 3 Student Training**: It then train student models (Smaller: 25M, Base: 97M, Large: 335M parameters) on pseudo-labeled real data. This stage produces result containing realistic, robust depth estimation that generalizes well.

**Stage 4 Metric Finetuning:** This is optional stage. It is further fine-tune on metric-labeled datasets (e.g., NYUv2, KITTI, Virtual KITTI). This enables accurate absolute depth output.

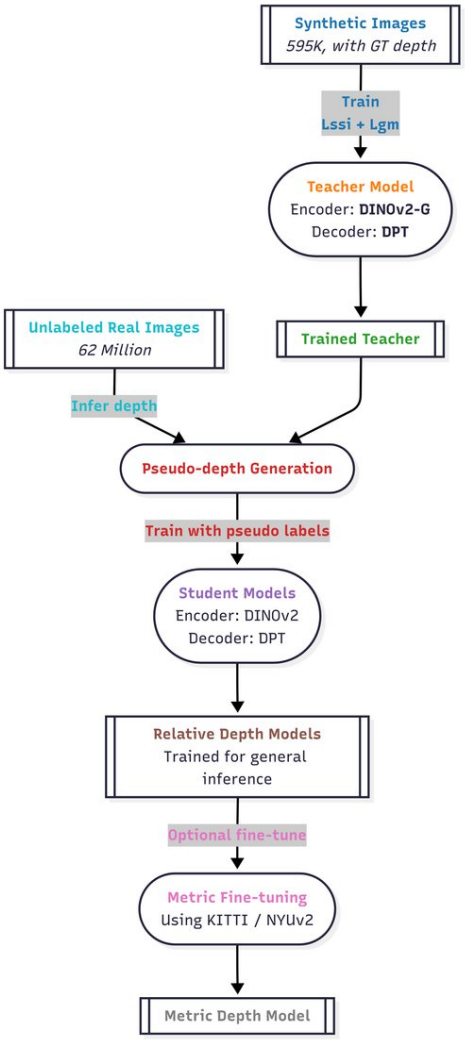


*Figure 1.6:Depth Anything V2 Training Framework Flow. Source: Roboflow*

## 1.5 ORB-SLAM

### 1.5.1 How it works ?

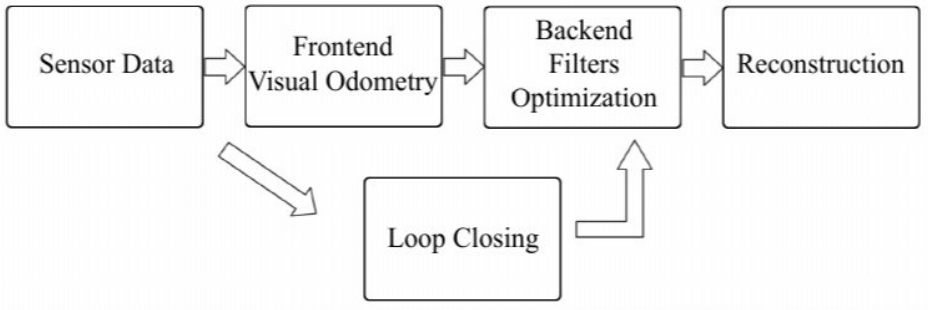


*Figure 1.7: SLAM pipeline. [7]*

A typical visual SLAM workflow includes the following steps:

- **Sensor data acquisition**: In visual SLAM, this mainly refers to acquisition and preprocessing of camera images. The camera can be roughly divided into three categories: a monocular camera has only one camera; a stereo camera has two; RGB-D camera collects color images and measure the distance of the scene from the camera for each pixel.
- **Visual Odometry (VO)**: Estimate the camera movement between adjacent frames (ego-motion) and generate a rough local map. VO is also known as the frontend.
- **Loop Closing**: Loop closing determines whether the robot has returned to its previous position in order to reduce the accumulated drift. If a loop is detected, it will provide information to the backend for further optimization.
- **Backend filtering/optimization**: The backend receives camera poses at different time stamps from VO and results from loop closing, and then applies optimization to generate a fully optimized trajectory and map. Because it is connected after the VO, it is also known as the backend.
- **Reconstruction**: It constructs a task-specific map based on the estimated camera trajectory.

This project will choose **ORB-SLAM** as a feature-based monocular SLAM system that is extract and match image feature points, and then estimate the camera motion and scene structure between two frames. This system has the capable of operating in real time, in small and large, indoor and outdoor environments.

### 1.5.2 ORB feature:

ORB features are also composed of two parts: ORB key points and ORB descriptor. Its key point is called “oriented FAST”, which is an improved version of the FAST (*Features from Accelerated Segment Test*). We will introduce what the FAST corner below is. Its descriptor is called BRIEF (*Binary Robust Independent Elementary Feature*). Therefore, the extraction of ORB features is divided into the following two steps:

1) **FAST corner point extraction**: find the corner point in the image. Compared with the original FAST, the main direction of the feature points is calculated in ORB, making the subsequent BRIEF descriptor rotation-invariant.
2) **BRIEF descriptor**: describe the surrounding image area where the feature points were extracted in the previous step. ORB has made some improvements to BRIEF, mainly referring to utilizing the previously calculated direction.

#### 1.5.2.1 FAST key point:

FAST is a kind of corner point, which mainly detects the local grayscale changes and is known for its fast speed. Its main idea is: if a pixel is very different from the neighboring pixels (too bright or too dark), it is more likely to be a corner point. Compared with other corner detection algorithms, FAST only needs to compare the pixels' brightness. Its entire procedure is as follows:

1) Select pixel $p$ in the image, assuming its brightness as $I_p$
2) Set a threshold $T$ (for example, 20% of $I_p$)
3) Take the pixel $p$ as the center, and select the 16 pixels on a circle with a radius of 3.
4) If there are consecutive $N$ points on the selected circle whose brightness is greater than $I_p + T$ or less than $I_p - T$, then the central pixel p can be considered a feature point. $N$ usually takes 12, which is FAST-12. Other commonly used $N$ values are 9 and 11, called FAST-9 and FAST-11, respectively).
5) Iterate through the above four steps on each pixel.

The original FAST corners are often clustered, meaning a lot of FAST corners may present in the same area. Therefore, after the initial detection, non maximal suppression is required. Only corner points with the maximum response in a specific location will be retained to avoid the corners concentrating. In Figure 1.5, we show the result of extracting feature from an image.

*Figure 1.8: FAST features. [7]*

The calculation of FAST feature points only compares the brightness difference between pixels, thus the speed is very fast, but it suffers from lousy repeatability and uneven distribution. Moreover, FAST corner points do not include direction information. Because it fixed the radius of the circle as 3, there is also a scaling problem: a place that looks like a corner from a distance may not be a corner when it comes close. To solve those, ORB adds the description of scale and rotation. The scale invariance is achieved by the image pyramid 3 and detect corner points on each layer. The rotation of features is realized by the intensity centroid method.

An image pyramid is a common approach in computer vision. The bottom of the pyramid is the original image. For each layer up, the image is scaled with a fixed ratio so that we have images of different resolutions. The smaller image can be seen as a scene viewed from a distance.

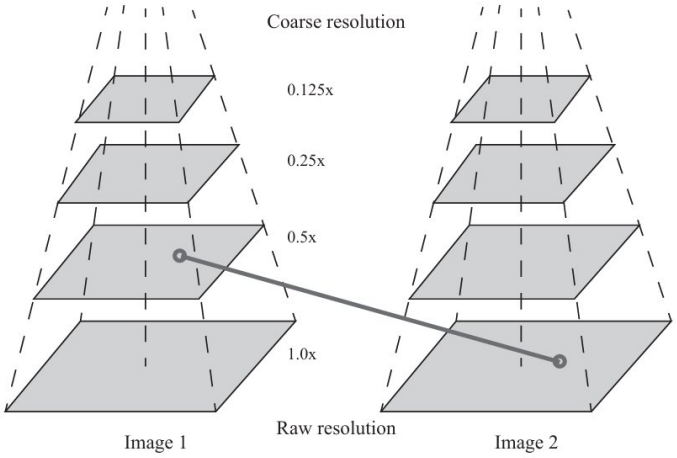


*Figure 1.9: Use pyramids to match images at different resolutions. [7]*

In the feature matching we can match images on different layers to achieve scale invariance. For example, if the camera moves backward, we should find a match in the upper layer of the previous image pyramid and the lower layer of the next image.

In terms of rotation, we calculate the gray centroid of the image near the feature point. The so-called centroid refers to the gray value of the image block as the center of weight. The specific steps are as follows:

1) In a small image block $B$, define the moment of the image block as:

$$m_{pq} = \sum_{x,y \in B} x^p y^q I(x,y); p,q = \{0,1\}.$$

2) Calculate the centroid of the image block by the moment:

$$C = \left( \frac{m_{10}}{m_{00}}, \frac{m_{01}}{m_{00}} \right).$$

3) Connect the geometric center $O$ and the centroid $C$ of the image block to get a direction vector $\overrightarrow{OC}$, so the direction of the feature point can be defined as:

$$\theta = \arctan\left(\frac{m_{10}}{m_{00}}\right).$$

FAST corner points have a description of scale and rotation, which significantly improves the robustness of their representation between different images. This improved FAST is called oriented FAST in ORB.

#### 1.5.2.2 BRIEF Descriptor:

After extracting the Oriented FAST key points, we calculate the descriptor for each point. ORB uses an improved BRIEF feature description. Let's first introduce what BRIEF is.

BRIEF is a binary descriptor. Its description vector consists of many zeros and ones, which encode the size relationship between two random pixels near the key point (such as $p$ and $q$): If $p$ is greater than $q$, then take 1, otherwise take 0. If we take 128 such $p$, $q$ pairs, we will finally get a 128-dimensional vector consisting of 0s and 1s. The BRIEF implements the comparison of randomly selected points, which is very fast. Since it expresses in binary, it is also very convenient to store and suitable for real-time image matching.

The original BRIEF descriptor does not have rotation invariance, so it is easy to get lost when the image is rotated. The ORB calculates the direction of the key points in the FAST feature point extraction stage. The direction information can be used to calculate the Steer BRIEF feature after the rotation so that the ORB descriptor has better rotation invariance. Due to the consideration of rotation and scaling, the ORB performs well under translation, rotation, and scaling. Meanwhile, the combination of FAST and BRIEF is very efficient, which makes ORB features very popular in real-time SLAM. In the following, we will move on to feature matching between different images.

#### 1.5.2.3 Feature matching:

Broadly speaking, feature matching determine the view correspondence between the landmarks (feature points) and the landmarks (feature points) seen before. By accurately matching the descriptors between images or between images and maps, we can reduce a lot of work for subsequent pose estimation and optimization operations.

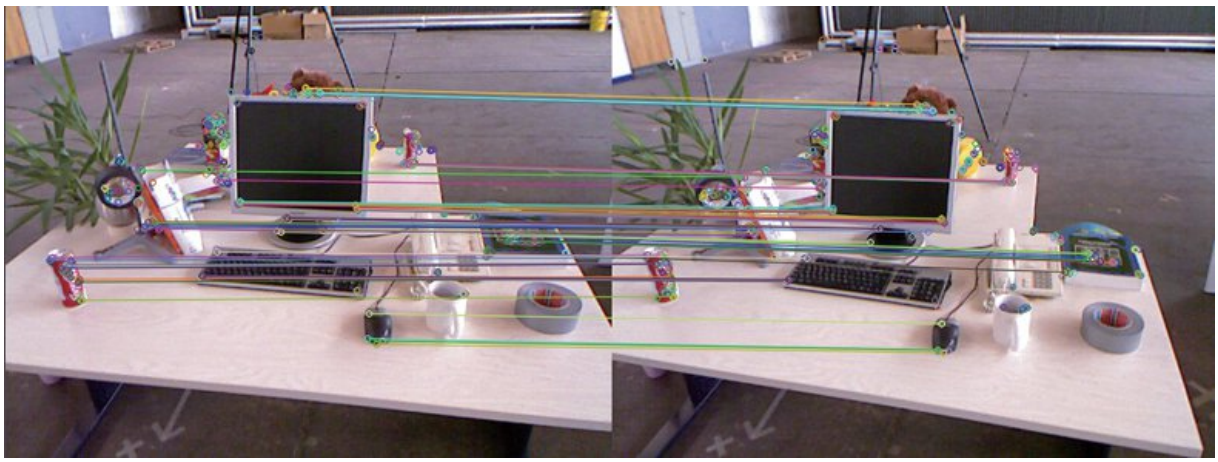

*Figure 1.10: ORB Feature mathing between 2 images. [7]*

However, due to the locality of image features, mismatches are common and have not been effectively resolved for a long time. Consider images at two moments. Assume features $x_t^m$, m = 1, 2,..., M are extracted in the image $I_t$; and $x_{t+1}^n$, $n$ = 1,2,...,N in $I_{t+1}$ .The simplest feature matching method is the brute-force matcher, which measures the distance between each pair of the features $x_t^m$ and all $x_{t+1}^n$ descriptors. Then sort the matching distance, and take the closest one as the matching point.

For binary descriptors (such as BRIEF), we often use Hamming distance as a metric. The Hamming distance between two binary vectors refers to the number of different digits.

### 1.5.3 How to represent a 3d rigid body motion in space:

In VSLAM are the problem of the camera's position and orientation. This consists of one rotation plus one translation, the translation part does not really have any problems, but the rotation part is questionable.

A vector $\vec{p}$ in the camera system may have coordinates $p_c$; and in the world coordinate system, its coordinates may be $p_w$ . Then what is the conversion between these two coordinates? It is necessary to first obtain the coordinate values of the point in the camera system and then use the transform rule to do the coordinate transform. We can describe it with a transform matrix $\boldsymbol{T}$.

Intuitively, the motion between two coordinate systems consists of a rotation plus a translation, which is called rigid body motion. Suppose we have a unit-length orthogonal base ($\boldsymbol{e}_1, \boldsymbol{e}_2, \boldsymbol{e}_3$). After a rotation it becomes ($\boldsymbol{e}_1', \boldsymbol{e}_2', \boldsymbol{e}_3'$). Then, for the same vector $\vec{a}$ (the vector does not move with the rotation of the coordinate system), its coordinates in these two coordinate systems are $[a_1, a_2, a_3]^T$ and $[a_1', a_2', a_3']^T$.

$$\begin{bmatrix} a_1 \\ a_2 \\ a_3 \end{bmatrix} = \begin{bmatrix} \boldsymbol{e}_1^T\boldsymbol{e}_1' & \boldsymbol{e}_1^T\boldsymbol{e}_2' & \boldsymbol{e}_1^T\boldsymbol{e}_3' \\ \boldsymbol{e}_2^T\boldsymbol{e}_1' & \boldsymbol{e}_2^T\boldsymbol{e}_2' & \boldsymbol{e}_2^T\boldsymbol{e}_3' \\ \boldsymbol{e}_3^T\boldsymbol{e}_1' & \boldsymbol{e}_3^T\boldsymbol{e}_2' & \boldsymbol{e}_3^T\boldsymbol{e}_3' \end{bmatrix} \begin{bmatrix} a_1' \\ a_2' \\ a_3' \end{bmatrix} \triangleq \boldsymbol{R}\boldsymbol{a}'.$$

To describe the relationship between the two coordinates, define it as a matrix $\boldsymbol{R}$. This matrix consists of the inner product between the two sets of bases, describing the same vector's coordinate transformation relationship before and after the rotation. As long as the rotation is the same, this matrix is the same. So we call it the **rotation matrix**.

In practice, we may define the coordinate system 1 and 2, then the vector a under the two coordinates is $\boldsymbol{a}_1$, $\boldsymbol{a}_2$ . The relationship between the two systems should be:

$$\boldsymbol{a}_1 = \boldsymbol{R}_{12}\boldsymbol{a}_2 + \boldsymbol{t}_{12}.$$

$\boldsymbol{R}_{12}$ means "rotation of the vector from system 2 to system 1" and $\boldsymbol{t}_{12}$ , readers may just take it as a translation vector. It corresponds to a vector from the system 1's origin pointing to system 2's origin, and the coordinates are taken under system 1. Understand it as "a vector from 1 to 2".

This form is not elegant after multiple transformations. Therefore, we introduce homogeneous coordinates and transformation matrices, rewriting the form:

$$\begin{bmatrix} \boldsymbol{a}' \\ 1 \end{bmatrix} = \begin{bmatrix} \boldsymbol{R} & \boldsymbol{t} \\ \boldsymbol{0}^T & 1 \end{bmatrix} \begin{bmatrix} \boldsymbol{a} \\ 1 \end{bmatrix} \triangleq \mathbf{T} \begin{bmatrix} \boldsymbol{a} \\ \boldsymbol{1} \end{bmatrix}.$$

We add 1 at the end of a 3D vector and turn it into a 4D vector called homogeneous coordinates. Then, we can write the rotation and translation in one matrix. The matrix $\boldsymbol{T}$ is called transform matrix. Matrix $\boldsymbol{T}$ has a special structure: the upper left corner is the rotation matrix, the right side is the translation vector, the lower-left corner is 0 vector, and the lower right corner is 1.

### 1.5.4 Optimization

A basic SLAM problem: how to solve the estimated $\boldsymbol{x}$ (localization) and $\boldsymbol{y}$ (mapping) problem with the noisy control input $\boldsymbol{u}$ and the sensor reading $\boldsymbol{z}$ data? We model the SLAM problem as a state estimation problem: How to estimate the internal, hidden state variables through the noisy measurement data?

Even if we have a high-precision camera and controller, the motion and observations equations can only be approximated. Therefore, instead of assuming that the data must conform to the equation precisely, it is better to find an estimation approach to get the state from the noisy data. This section will introduce the basic unconstrained nonlinear optimization method.

#### 1.5.4.1 Non-least-squares optimization

Consider a simple least-square problem:

$$\min_x F(\boldsymbol{x}) = \frac{1}{2}\|f(\boldsymbol{x})\|_2^2.$$

Let the derivative of the objective function be zero, and then find the optimal value of $x$, just like finding the extreme value of a scalar function:

$$\frac{dF}{d\boldsymbol{x}} = \mathbf{0}.$$

But, some derivative functions may be complicated in form, making the equation difficult to solve. For the least-square problem that is inconvenient to solve directly, we can use the iterated methods to start from an initial value and continuously update the current estimations to reduce the objective function. The specific steps can be listed as follows:

1) Give an initial value $\mathrm{x}_0$ .
2) For $k$-th iteration, we find an incremental value of $\Delta\mathbf{x}_\mathrm{k}$ , such that the object function $\left\|f(\boldsymbol{x}_k + \Delta\boldsymbol{x}_k)\right\|_2^2$ reaches a smaller value.
3) If $\Delta\mathbf{x}_\mathrm{k}$ is small enough, stop the algorithm.
4) Otherwise, let $\mathbf{x}_{\mathrm{k}+1} = \mathbf{x}_\mathrm{k} + \Delta\mathbf{x}_\mathrm{k}$ and return to step 2.

When the function decreases until the increment becomes very small, it is considered that the algorithm converges, and the objective function reaches a minimum value. In this process, the problem is how to find the increment at each iteration point. We examine how to find this increment $\Delta x$.

**1.5.4.2 The Gauss-Newton Method:**

Its idea is to carry out a first-order Taylor expansion of $f(x)$.

$$f(\boldsymbol{x} + \Delta\boldsymbol{x}) \approx f(\boldsymbol{x}) + \boldsymbol{J}(\boldsymbol{x})^T \Delta\boldsymbol{x}.$$

Here $J(x)$ is the derivative of $f(x)$ with respect to $x$, which is a column vector of $n \times 1$. The current goal is to find the increment $\Delta x$ such that $\left\|f(\boldsymbol{x} + \Delta\boldsymbol{x})\right\|^2$ reached the minimum. In order to find $\Delta x$, we need to solve a linear least-square problem:

$$\Delta\boldsymbol{x}^* = \underset{\Delta\boldsymbol{x}}{\operatorname{argmin}} \frac{1}{2}\left\|f(\boldsymbol{x}) + \boldsymbol{J}(\boldsymbol{x})^T \Delta\boldsymbol{x}\right\|^2.$$

Find the derivative of the above formula with respect to $\Delta x$ and set it to zero, we get:

$$\boldsymbol{J}(\boldsymbol{x}) f(\boldsymbol{x}) + \boldsymbol{J}(\boldsymbol{x}) \boldsymbol{J}^T(\boldsymbol{x}) \Delta\boldsymbol{x} = 0.$$

The following equation can be obtained:

$$\underbrace{\mathbf{J}(\mathrm{x})\,\mathbf{J}^T}_{\mathbf{H}(\mathrm{x})}(\mathrm{x})\,\Delta\mathrm{x} = \underbrace{-\mathbf{J}(\mathrm{x})\,f(\mathrm{x})}_{\mathrm{g}(\mathrm{x})}$$

This equation is a linear equation of the variable $\Delta x$. We call it normal equation or Gauss-Newton equation. We define the coefficients on the left as **H** and the coefficient on the right as **g**, then the above formula becomes:

$$\mathbf{H}\Delta x_k = \mathbf{g}$$

Solving the normal equation is the core of the entire optimization problem. If we can get the $\Delta x$ in each iteration, then the algorithm of the Gauss-Newton method can be written as:

1) Set it initial value as $x_0$.
2) For $k$-th iteration, calculate the Jacobian $J(x_k)$ and residual $f(xk)$.
3) Solve the normal equation: $\mathbf{H}\Delta x_k = \mathbf{g}$.
4) If $\Delta x_k$ is small enough, stop the algorithm. Otherwise let $x_{k+1} = x_k + \Delta x_k$ and return to step 2.

In practice, we can't guarantee that the Gauss-Newton method converges. Sometimes they can reach even larger objective function values than the initial one. The Levenberg-Marquardt method corrects these problems to a certain extent. It is generally considered more robust than the Gauss-Newton method, but its convergence rate may be slower than the Gauss-Newton method. Such a kind of method is also called as damped Newton Method.

### 1.5.4.3 The Levernberg-Marquatdt Method:

The approximate second-order Taylor expansion used in the Gauss-Newton method can only have a good approximation effect near the expansion point. So, a range should be added to $\Delta x$, called trust-region. This range defines under what circumstances the second-order approximation is valid.

To determine the scope of this trust-region is to based on the difference between our approximate model and the real object function: if the difference is small, it means that the approximation is good, and we may expand the trust-region; conversely, if the difference is large, we will reduce the range of approximation.

Define an indicator $\rho$ to describe the degree of approximation:

$$\rho = \frac{f(\boldsymbol{x} + \Delta \boldsymbol{x}) - f(\boldsymbol{x})}{\boldsymbol{J}(\boldsymbol{x})^T \Delta \boldsymbol{x}}$$

The numerator of $\rho$ is the decreasing value of the real object function, and the denominator is the decreasing value of the approximate model. If $\rho$ is close to 1, we may say the approximation is good. A small $\rho$ is indicating that the actual reduced value is far less than the approximate reduced value. The approximation is poor in this case, and the trust-region needs to be reduced. Conversely, if $\rho$ is relatively large, it means that the actual decline is greater than expected, and we can enlarge the approximate range.

Here is an improved version of the nonlinear optimization framework, which will have a better effect than the Gauss-Newton method:

1) Give the initial value $x_0$ and initial trust-region radius $\mu$.
2) For $k$-th iteration, we solve a linear problem based on Gauss-Newton's method added with a trust-region:
3) $\min_{\Delta x_k} \frac{1}{2} \left\| f(x_k) + J(x_k)^T \Delta x_k \right\|^2, \ s.t. \ \ \|D\Delta x_k\|^2 \le \mu.$
4) where $\mu$ is the radius and **D** is a coefficient matrix, which will be discussed in below.
5) Compute $\rho$ using equation.
6) If $\rho > \frac{3}{4}$, set $\mu = 2\mu$.
7) Otherwise, if $\rho < \frac{1}{4}$, set $\mu = 0.5\mu$.
8) If $\rho$ is larger than a given threshold, set $x_{k+1} = x_k + \Delta x_k$.
9) Go back to step 2 if not converged, otherwise return the result.

#### 1.5.4.4 Bundle adjustment:

The so-called Bundle Adjustment or **BA** refers to optimizing both camera parameters (intrinsic and extrinsic) and 3D landmarks with images. Consider the bundles of light rays emitted from 3D points. They are projected into the image planes of several cameras and then detected as feature points. The purpose of optimization can be explained as to adjust the camera poses and the 3D points, to ensure the projected 2D features (bundles) match the detected results using nonlinear optimization method describe above.

Suppose we have landmarks 3D locations $\boldsymbol{X}_{w,j} \in \mathbb{R}^{3\times3}$ and keyframe poses matrix $\boldsymbol{T}_{iw} \in \mathbb{R}^{4\times4}$, where $w$ stands for the world reference, are optimized minimizing the reprojection error with respect to the matched keypoints $x_{i,j} \in \mathbb{R}^2$. The error term for the observation of a map point $j$ in a keyframe $i$ is:

$$e_{i,j} = x_{i,j} - f_i(\boldsymbol{T}_{iw}, \boldsymbol{X}_{w,j})$$

Where $f_i$ is the projection error:

$$f_i\left(\boldsymbol{T}_{iw}, \boldsymbol{X}_{w,j}\right) = \begin{bmatrix} f_{i,u}\dfrac{x_{i,j}}{z_{i,j}} + c_{i,u} \\ f_{i,v}\dfrac{x_{i,j}}{z_{i,j}} + c_{i,v} \end{bmatrix}$$

$$\begin{bmatrix} x_{i,j} & y_{i,j} & z_{i,j} \end{bmatrix}^T = \boldsymbol{R}_{iw}\boldsymbol{W}_{w,j} + \boldsymbol{t}_{iw}$$

Where $\boldsymbol{R}_{iw} \in \mathbb{R}^{3\times3}$ and $\boldsymbol{t}_{iw} \in \mathbb{R}^3$ are respectively the rotation and translation part of $\boldsymbol{T}_{iw}$ ; $(f_{i,u}, f_{i,v})$ and $(c_{i,u}, c_{i,v})$ are the focal length and principle point associated to camera $i$. The cost function to be minimized is:

$$C = \sum_{i,j} \rho_h(\boldsymbol{e}_{i,j}^T \boldsymbol{\Omega}_{i,j}^{-1} \boldsymbol{e}_{i,j})$$

Where $\rho_h$ is the Huber robust cost function and $\boldsymbol{\Omega}_{i,j} = \sigma_{i,j}^2 \boldsymbol{I}_{2\times 2}$ is the covariance matrix associated to the scale at which the keypoint was detected.

In case of full BA all points and keyframes are optimized, by the exception of the first keyframe which remain fixed as the origin. In local BA all points included in the local are are optimized, while a subset of keyframes is fixed. In pose optimization, or motion-only BA, all points are fixed and only the camera pose is optimized.

**1.5.4.5 Pose graph optimization:**

Landmarks occupy most of the optimization time. In fact, after several observations, the spatial position of the landmarks will converge to a value and remain almost unchanged, while the divergent outliers are usually invisible. Optimizing the convergent landmarks seems a bit useless.

We can construct a graph optimization with only pose variables. The edge between the pose vertices can be set with measurement by the ego-motion estimation obtained from feature matching. The difference is that once the initial estimation is completed, we no longer optimize the positions of those landmark points but onl y care about the connections between all camera poses. In this way, we save a lot of calculations for landmarks optimization and only keep the keyframe's trajectory, thus constructing the so-called Pose Graph.

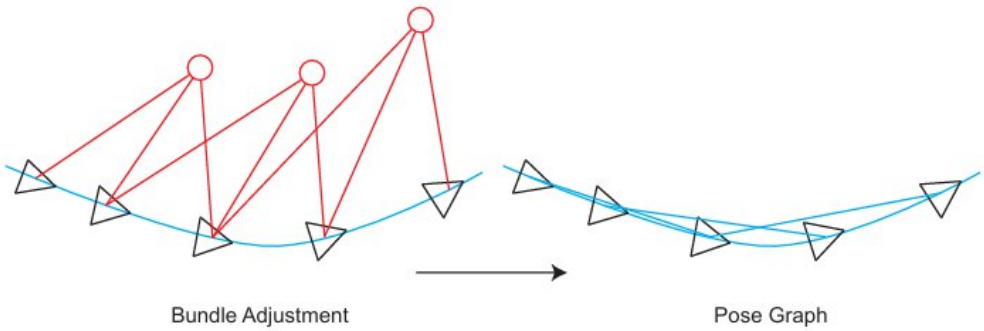


*Figure 1.11: Pose Graph diagram.*

*When we no longer optimize the landmark in Bundle Adjustment, and only regard them as constraints on the pose nodes, we get a pose graph with a much reduced calculation scale. [7]*

Given a pose graph of binary edges (see …) we define the error in an edge as:

$$e_{i,j} = log_{Sim(3)}\left(\boldsymbol{S}_{ij}\boldsymbol{S}_{jw}\boldsymbol{S}_{iw}^{-1}\right)$$

Where $\boldsymbol{S}_{ij} \in \mathbb{R}^{4\times 4}$ is the relative similarity transformation between both keyframes computed just before the pose graph optimizatio, in the case of the loop closure edge this relative transformation is computed to find the best rotation and translation that aligns the two sets of points; $\boldsymbol{S}_{iw}, \boldsymbol{S}_{jw} \in \mathbb{R}^{4\times 4}$ is the current guess of pose $i$ and $j$

respectively. The result give a matrix $\mathbb{R}^{4\times4}$ so $log_{Sim(3)}$ converts the matrix difference into a 7-number vector: 3 for rotation, 3 for translation, 1 for scale.

The goal is to optimize the Sim(3) keyframe poses minimizing the cost function:

$$C = \sum_{i,j} (\boldsymbol{e}_{i,j}^T \Lambda_{i,j} \boldsymbol{e}_{i,j})$$

Where $\Lambda_{i,j}$ is the information matrix of the edge, which set to the identity. This cost measures how wrong the whole pose graph is that add up all the error of the relative pose between i and j since information matrix is identity. Meaning it is the sum of $\left\|\boldsymbol{e}_{i,j}\right\|^2$.

# CHAPTER 2: IMPLEMENTATION

## 2.1 System overview:

Our system incorporates 3 main processes marking in gray as show in Figure 2.1

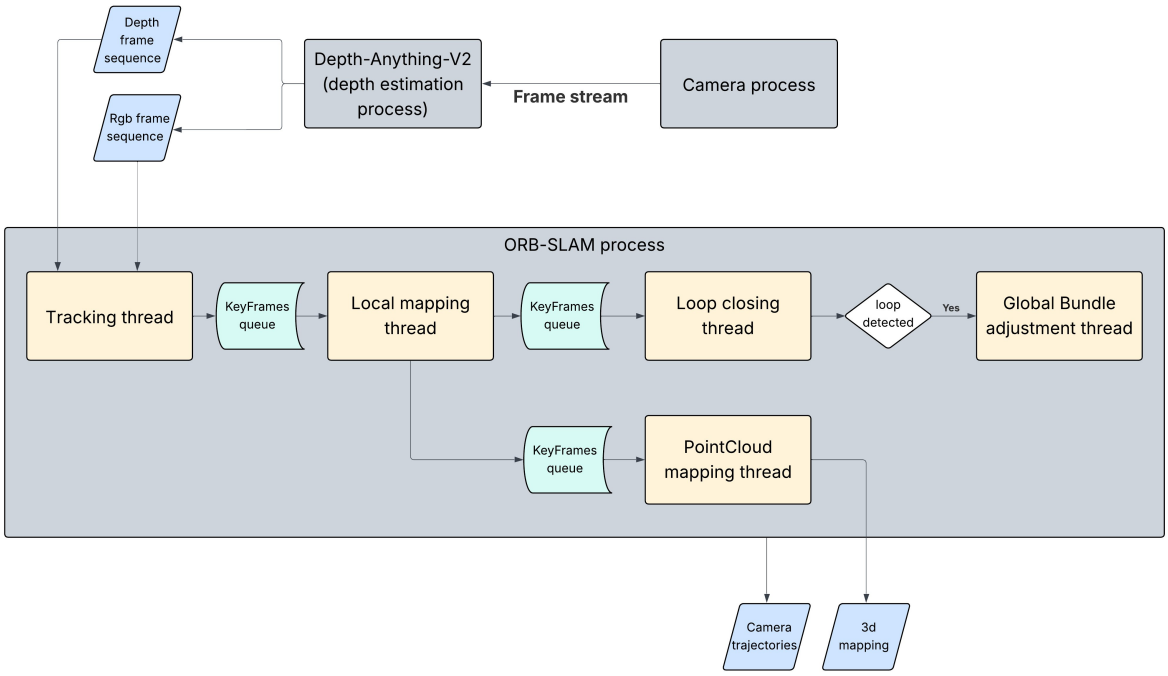


*Figure 2.1: System overall pipeline.*

The system start up by receiving frame stream from a phone camera, for visual SLAM before system initializing, the camera must be first calibrated to obtain its intrinsic parameters using a 9x6 chessboard-like as shown in Figure 2.2.

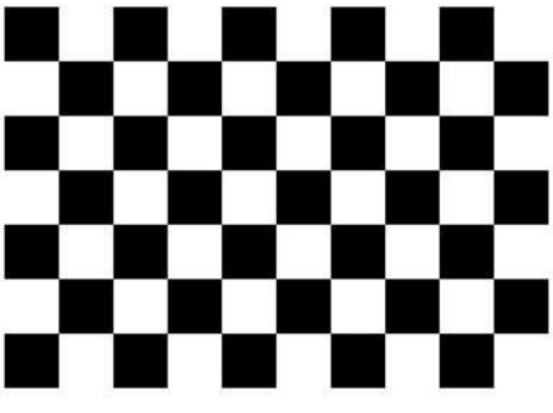

*Figure 2.2: 9x6 Calibration board for finding camera's instrinsic parameters.*[1]

As depth estimation process receive stream from camera, we proceed to capture frames from the stream at 30Hz frequency. At the same time, capture frames will be

[1] https://github.com/opencv/opencv/blob/4.x/doc/pattern.png

processed by deep learning model to yield out the corresponding depth maps as 16 bit grayscale images (the further the object, the brighter it become), shown in Figure 2.3.

*Figure 2.3: RGB image and its corresponding depth image.*

After finishing process a sequence of frame and the corresponding depth maps, ORB-SLAM process will be started up to begin tracking the camera motion and building a model of the enviroment that exist in those frames.

ORB-SLAM is divided into 4 main threads:

- The main **tracking** thread is in charge of localizing the camera with every frame and deciding when to insert a new keyframe. It performs first an initial feature matching with the previous frame and optimize the pose using motion-only BA. If the tracking is lost (e.g. due to occlusions or abrupt movements), the place recognition module is used to perform a global relocalization.
- The **local mapping** processes new keyframes and performs local BA to achieve an optimal reconstruction in the sur roundings of the camera pose. An exigent point culling policy is applied in order to retain only high quality points. The local mapping is also in charge of culling redundant keyframes.
- The **loop closing** searches for loops with every new keyframe. If a loop is detected, it compute a similarity transformation that informs about the drift accumulated in the loop. Then both sides of the loop are aligned and duplicated points are fused. Finally a launch of **global BA** thread to optimize all keyframes and points in the map, except the origin keyframe to achieve the optimal solution
- The **pointcloud mapping** thread receive new keyframe from local mapping thread and using its associating depth map to display its pointcloud representation. Then accumulating into a global pointcloud for a full 3d enviroment mapping.

The rest of this chapter, we go into detail of each thread module implementation.

## 2.2 Tracking:

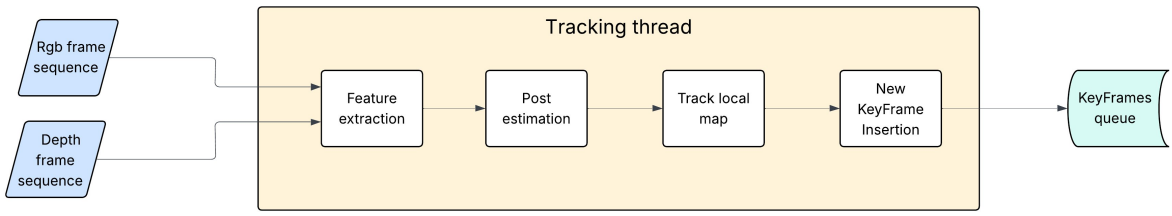


*Figure 2.4: Tracking pipeline.*

### 2.2.1 Feature extraction:

They originally extracted FAST corners at 8 scale levels with a scale factor of 1.2. For image resolutions from 512x384 to 752x480 pixels they extract 1000 corners, for higher resolutions, they extracted 2000 corners (but we increase to 1000 corners more and 9 scale levels for noisy data).

In detail, for an image, they divided each scale level in a grid, trying to extract at least 5 corners per cell. Then detecting corners in each cell, adapting the detector threshold if not enough corners are found. The amount of corners retained per cell is also adapted if some cells contains no corners (textureless or low contrast). The orientation and ORB descriptor are then computed on the retained FAST corners. The ORB descriptor is used in all feature matching.

### 2.2.2 Pose estimation:

If tracking was successful for last frame, the system used a constant velocity motion model (The camera keeps moving the same amount every frame, no sudden stops or jumps (unless tracking fails). So if the camera moved: between frame $t-1$ and frame $t$, then the system predicts that the camera will do the same thing from frame $t$ to frame $t+1$.) to predict the camera pose and perform a guided search (with a motion model, ORB-SLAM predicts where each map point should appear in the next frame. It only searches near that predicted location) of the map points observed in the last frame. If not enough matches were found, it used a wider search of the map points around their position in the last frame. The pose is then optimized with the found correspondences.

If the tracking is lost, it converted the frame into bag of words and query the recognition database for keyframe candidates for global relocalization. It computes correspondences with ORB associated to map points in each keyframe. It then perform alternatively RANSAC iterations for each keyframe and try to find a camera pose using the PnP algorithm which we explained in APPENDIX If a camera pose with enough inliers found, it optimize the pose and perform a guided search of more matches with the map points of the candidate keyframe. Finally the camera pose is again optimized, and if supported with enough inliers, tracking procedure continues.

### 2.2.3 Track local map:

Once the system has an estimation of the camera pose and an initial set of feature matches, it can project the map into the frame and search more map point correspondences.

To bound the complexity in large maps, it only project a local map. This local map contains the set of keyframes $K_1$, that share map points with the current frame, and a set $K_2$ with neighbors to the keyframes $K_1$ in the covisibility graph.

The local map also has a reference keyframe $K_{ref} \in K_1$ which shares most map points with current frame. Now each map point seen in $K_1$ and $K_2$ is searched in the current frame as follows:

1) Compute the map point projection $x$ in the current frame. Discard if it lays out of the image bounds.
2) Compute the angle between the current viewing ray $v$ and the map point mean viewing direction $n$. Discard if $v \cdot n < \cos 60^o$
3) Compute the distance $d$ from map point to camera center. Discard if it is out of the scale invariance region of the map point $d \notin [d_{min}, d_{max}]$.
4) Compute the scale in the frame by the ratio $\frac{d}{d_{min}}$.
5) Compare the representative descriptor $D$ of the map point with the still unmatched ORB features in the frame, at the predicted scale, and near $x$, and associate the map point with the best match.

The camera pose is finally optimized with all the map points found in the frame.

### 2.2.4 New keyframe insertion:

The last step is to decide if the current frame is a new keyframe. To insert a new keyframe all the following conditions must be met:

1) More than 20 frames must have passed from the last global relocalization.
2) Local mapping is idle, or more than 20 frames have passed from last keyframe insertion.
3) Current frame tracks at least 50 points.
4) Current frame tracks less than 90% points than $K_{ref}$.

Condition 4 impose a minimum visual change. Condition 1 ensures a good relocalization and condition 3 a good tracking. If a keyframe is inserted when the local mapping is busy (second part of condition 2), a signal is sent to stop local bundle adjustment, so that it can process as soon as possible the new keyframe.

## 2.3 Local mapping:

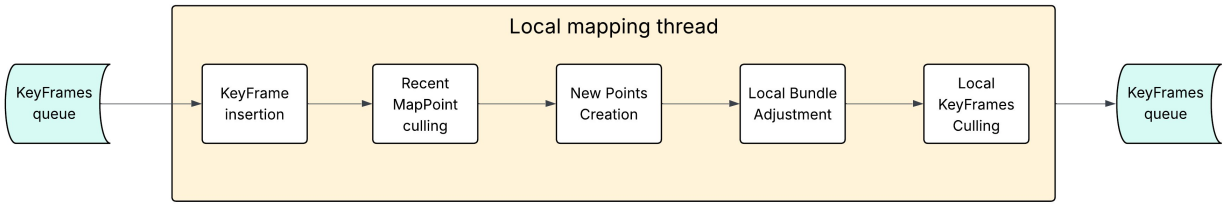


*Figure 2.5: Local mapping pipeline.*

### 2.3.1 KeyFrame Insertion:

At first the system update the covisibility graph, adding a new node for $K_i$ and updating the edges resulting from the shared map points with other keyframes then updating the spanning tree linking $K_i$ with the keyframe with most points in common. It then compute the bags of words representation of the keyframe, that will help in the data association for triangulating new points.

### 2.3.2 Map Points culling:

In order to retain map point in the map, it must pass a restrictive test during the first three keyframes after creation, that ensures that they are trackable and not wrongly triangulated. A point must fulfill these two conditions:

1) The tracking must find the point in more than the 25% of the frames in which it is predicted to be visible.
2) If more than one keyframe has passed from map point creation, it must be observed from at least three keyframes.

Once a map point have passed this test, it can only be removed if at any time it is observed from less than three keyframes. This can happen when keyframes are culled and when local bundle adjustment discards outlier observations. This policy makes our map contain very few outliers.

### 2.3.3 New Points Creation:

New map points are created by triangulating ORB from connected keyframes $K_c$ in the covisibility graph. For each unmatched ORB in $K_i$ the system searchs a match with other unmatched point in other keyframe. And discard those matches that do not fulfill the epipolar constraint.

ORB pairs are triangulated (which mean draw a ray from camera 1 through feature 1, draw a ray from camera 2 through feature 2 and see where they intersect that gives the 3D point.), and to accept the new points, positive depth in both cameras, parallax (how much the view changes when you move the camera; the more parallax, the easier it is to figure out how far objects are), reprojection error (If you project this new 3D point back into the images, it should land close to the original ORB feature location. If the error is big then it a bad triangulation.) and scale consistency (ORB

features have pyramid levels – image scales.The scale in both frames must make sense for the distance of the point.) are checked.

Initially a map point is observed from two keyframes but it could be matched in others, so it is projected in the rest of connected keyframes, and correspondences are searched.

### 2.3.4 Local BA:

The local BA optimizes the currently processed keyframe $K_i$, all the keyframes connected to it in the covisibility graph $K_c$, and all the map points seen by those keyframes. All other keyframes that see those points but are not connected to the currently processed keyframe are included in the optimization but remain fixed. Observations that are marked as outliers are discarded at the middle and at the end of the optimization.

### 2.3.5 Local KeyFrames Culling:

In order to maintain a compact reconstruction, the local mapping tries to detect redundant keyframes and delete them. This is beneficial as bundle adjustment complexity grows with the number of keyframes, but also because it enables lifelong operation in the same environment as the number of keyframes will not grow unbounded, unless the visual content in the scene changes.

It discard all the keyframes in Kc whose 90% of the map points have been seen in at least other three keyframes in the same or finer scale. The scale condition ensures that map points maintain keyframes from which they are measured with most accuracy, where keyframes were discarded after a process of change detection.

## 2.4 Loop closing:

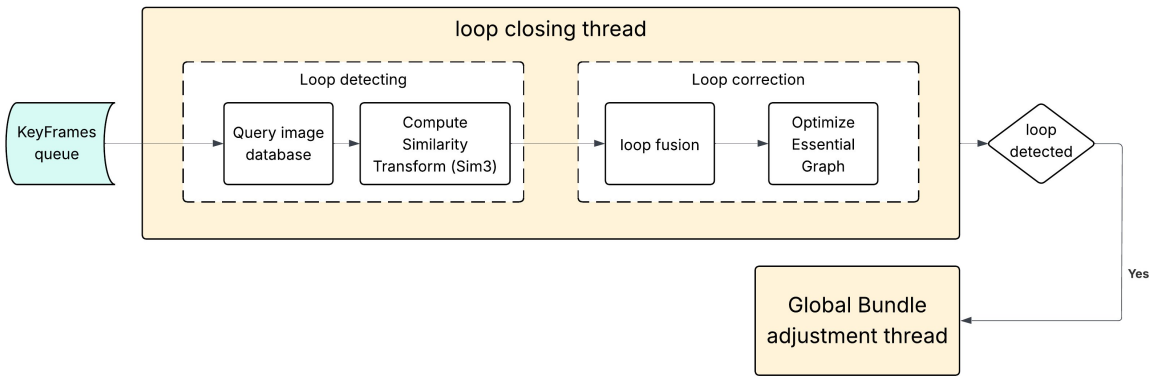


*Figure 2.6: Loop closing pipeline.*

The loop closing thread takes $K_i$, the last keyframe processed by the local mapping, and tries to detect and close loops.

### 2.4.1 Loop Candidates Detection:

At first it computes the similarity between the bag of words vector of $K_i$ and all its neighbors in the covisibility graph and retain the lowest score $s_{min}$. Then querying the recognition database and discard all those keyframes whose score is lower than a threshold.

In addition all those keyframes directly connected to $K_i$ are discarded from the results. To accept a loop candidate it must detect consecutively three loop candidates that are consistent (keyframes connected in the covisibility graph). There can be several loop candidates if there are several places with similar appearance to $K_i$.

### 2.4.2 Compute the Similarity Transformation:

In monocular SLAM there are seven degrees of freedom in which the map can drift, three translations, three rotations and a scale factor. Therefore to close a loop module need to compute a similarity transformation from the current keyframe $K_i$ to the loop keyframe $K_l$ that informs about the error accumulated in the loop.

The computation of this similarity will serve also as geometrical validation of the loop. Module first compute correspondences between ORB associated to map points in the current keyframe and the loop candidate keyframes.

At this point module has 3D to 3D correspondences for each loop candidate. It alternatively perform RANSAC iterations with each candidate, trying to find a similarity transformation using the method of Horn.

If it finds a similarity $S_{il}$ with enough inliers, it optimizes it, and performs a guided search of more correspondences. It optimize it again and, if $S_{il}$ is supported by enough inliers, the loop with $K_l$ is accepted.

### 2.4.3 Loop Fusion:

The first step in the loop correction is to fuse duplicated map points and insert new edges in the covisibility graph that will attach the loop closure. At first the current keyframe pose $T_{iw}$ is corrected with the similarity transformation $S_{il}$ and this correction is propagated to all the neighbors of $K_l$ (concatenating transformations), so that both sides of the loop get aligned.

All map points seen by the loop keyframe and its neighbors are projected into $K_l$ and its neighbors and matches are searched in a narrow area around the projection. All those map points matched and those that were inliers in the computation of $S_{il}$ are fused. All keyframes involved in the fusion will update their edges in the covisibility graph effectively creating edges that attach the loop closure.

### 2.4.4 Essential Graph Optimization:

To effectively close the loop, the module performs a pose graph optimization over the Essential Graph as define in APPENDIX, that distributes the loop closing error

along the graph. The optimization is performed over similarity transformations to correct the scale drift. The error terms and cost function are detailed in the APPENDIX.

After the optimization each map point is transformed according to the correction of one of the keyframes that observes it.

### 2.4.5 Global BA:

A full bundle adjustment (BA) optimization after the pose graph to achieve the optimal solution. This optimization can be very costly, so operation is performed in a separate thread, allowing the system to continue creating the map and detecting loops.

However, this introduces the challenge of merging the bundle adjustment output with the current map state. If a new loop is detected while the optimization is running, abort the optimization and proceed to close the loop, which will trigger the full BA optimization again.

Once the full BA finishes, they need to merge the updated subset of keyframes and points optimized by the full BA with the non-updated keyframes and points that were inserted while the optimization was running. This is accomplished by propagating the correction of the updated keyframes (i.e., the transformation from the non-optimized to the optimized pose) to the non-updated keyframes through the spanning tree. Non-updated points are transformed according to the correction applied to their reference keyframe.

## 2.5 Point Cloud Mapping:

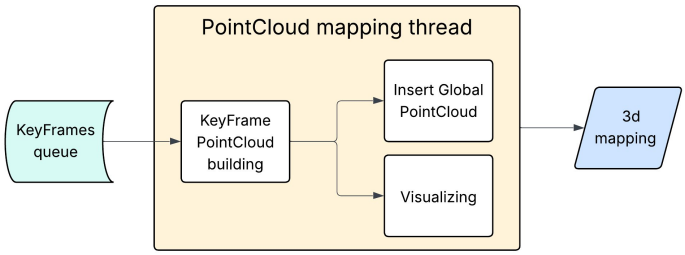


*Figure 2.7: PointCloud mapping pipeline.*

This module is an extension to the original ORB-SLAM2 system. For every new keyframe creation, its corresponding depth and rgb image got inserting into a queue and later pop out for building pointcloud and visualizing, then the pointcloud's frame got accumulate into a global pointcloud which represent 3d map that the camera observed. We don't implement any pointcloud fusion method.

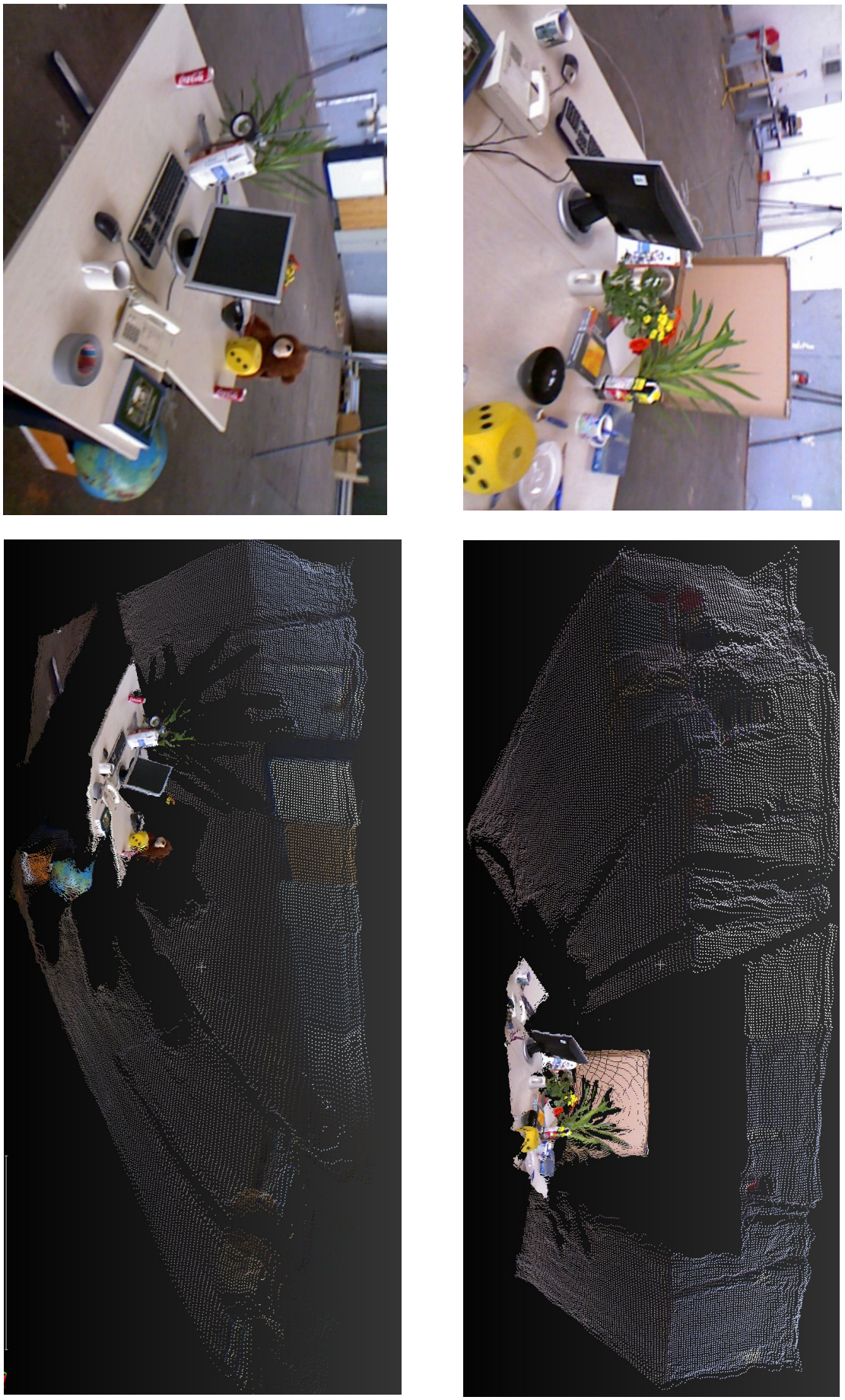

*Figure 2.8: PointCloud generated by the module.*

### 2.6 Libraries use:

Eigen:

- It is a C++ open-source linear algebra library. It provides fast linear algebra operations on matrices, as well as functions such as solving linear equations.
- We use Eigen to represent matrices and vectors and then extend to the calculation of rotation matrix and transformation matrix.

Sophus:

- Eigen provided geometry modules but did not support Lie algebra. A better Lie algebra library is the Sophus library maintained by Strasdat (https://github.com/strasdat/Sophus). The Sophus library supports SO(3) and SE(3).
- It is developed directly on top of Eigen.

Ceres:

- Google Ceres is a widely used optimization library for least-square problems.
- Ceres' GitHub address is (https://github.com/ceres-solver/ceres-solver).
- In Ceres, as users, we only need to define the optimization problem to be solved according to specific steps and then hand it over to the solver for calculation.

g2o:

- Another optimization library (widely used mainly in the SLAM field): g2o (general graphic optimization, g2o).
- GitHub: (https://github.com/RainerKuemmerle/g2o),
- It is a library based on graphic optimization. Graph optimization is a theory that combines nonlinear optimization with graph theory.

OpenCV:

- OpenCV provides many open-source image algorithms and is a very widely used image processing algorithm library in computer vision.

# CHAPTER 3: EXPERIMENT

We reassessed the performance of ORB-SLAM system with the replacement of RGBD camera sensor with monocular camera pipeling with Depth-Anything-V2 for predicting objects depth in order to track and construct 3d mapping.

We choose 2 set of data sources for evaluating monocular based camera. Both datasets were carried out with Intel(R) Core(TM) i5-7300HQ CPU @ 2.50GHz 2.50 GHz and 16.0 GB of RAM.

## 3.1 Dataset:

### 3.1.1 TUM RGB-D dataset:

This is the most compatible dataset with our current system configuration, rather than using camera sensor to generate depth field images, instead we try to use a deep learning model to generate them. The TUM RGB-D dataset [8] contains indoors sequences from RGB-D sensors grouped in several categories to evaluate object reconstruction and SLAM/odometry methods under different texture, illumination and structure conditions.

All sequences are recorded in 30 fps with image resolution of 640x480. The rgb images are in 8 bit format and depth field images are in 16 bit gray scale format (all pixels has a scale to convert to physical length measurement i.e 5000 = 1 meter).

After processing a sequence, system will output a file contain information of camera trajectory. Both the groundtruth and estimated trajectory to be in this format:

- The format of each line is '**timestamp tx ty tz qx qy qz qw**'
- **timestamp** (float) gives the number of seconds since the Unix epoch.
- **tx ty tz** (3 floats) give the position of the optical center of the color camera with respect to the world origin as defined by the motion capture system.
- **qx qy qz qw** (4 floats) give the orientation of the optical center of the color camera in form of a unit quaternion with respect to the world origin as defined by the motion capture system.
- The file may contain comments that have to start with "#".

### 3.1.2 KITTI dataset:

The KITTI dataset [9] contains stereo sequences recorded from a car in urban and high way environments. The stereo sensor has a 54 cm baseline and works at 10 Hz with a resolution after rectification of 1240x376 pixels. There are total of 22 sequences marking from 0 to 21 but only from 0 to 10 have their ground truth public.

Using stereo meaning sequences are captured with 2 monocular camera and objects depth are constructed from 2 images from left and right cameras. Since our configuration is only compatible with RGB-D sensor based, some modifications have been carried out for both input and output of the system. Ground truth sequence contains

a N x 12 table, where N is the number of frames of this sequence. Row i represents the i'th pose of the left camera coordinate system (i.e., z pointing forwards) via a 3x4 transformation matrix. The matrices are stored in row aligned order (the first entries correspond to the first row), and take a point in the i'th coordinate system and project it into the first (=0th) coordinate system. Hence, the translational part (3x1 vector of column 4) corresponds to the pose of the left camera coordinate system in the i'th frame with respect to the first (=0th) frame.

### 3.2 Metrics:

#### 3.2.1 Absolute Trajectory Error (ATE)[2]:

The absolute trajectory error directly measures the difference between points of the true and the estimated trajectory. Finally, the difference between each pair of poses are computed then output the mean/median/standard deviation of these differences. The ATE is well-suited for measuring the performance of visual SLAM systems.

#### 3.2.2 Relative Pose Error (RPE)[2]:

This metric computes the error in the relative motion between pairs of timestamps. The RPE is well-suited for measuring the drift of a visual odometry system.

### 3.3 Result:

#### 3.3.1 TUM RGB-D dataset:

*Table 2: ORB-SLAM evaluation under TUM dataset sequences.*

| Depth estimate method / Sequences | Depth-Anything-V2 | | RGB-D camera sensor | | Duration (second) |
|---|---|---|---|---|---|
| | ATE (m) | RPE (m) | ATE (m) | RPE (m) | |
| fr1_desk | 0.39 | 0.24 | 0.015 | 0.02 | 23 s |
| fr1_room | 0.7 | 0.21 | 0.09 | 0.04 | 50 s |
| fr3_no_structure_texture_near_with_loop | 0.38 | 0.09 | 0.018 | 0.015 | 56 s |

[2] We utilize tool from MichaelGrupp/evo: Python package for the evaluation of odometry and SLAM.

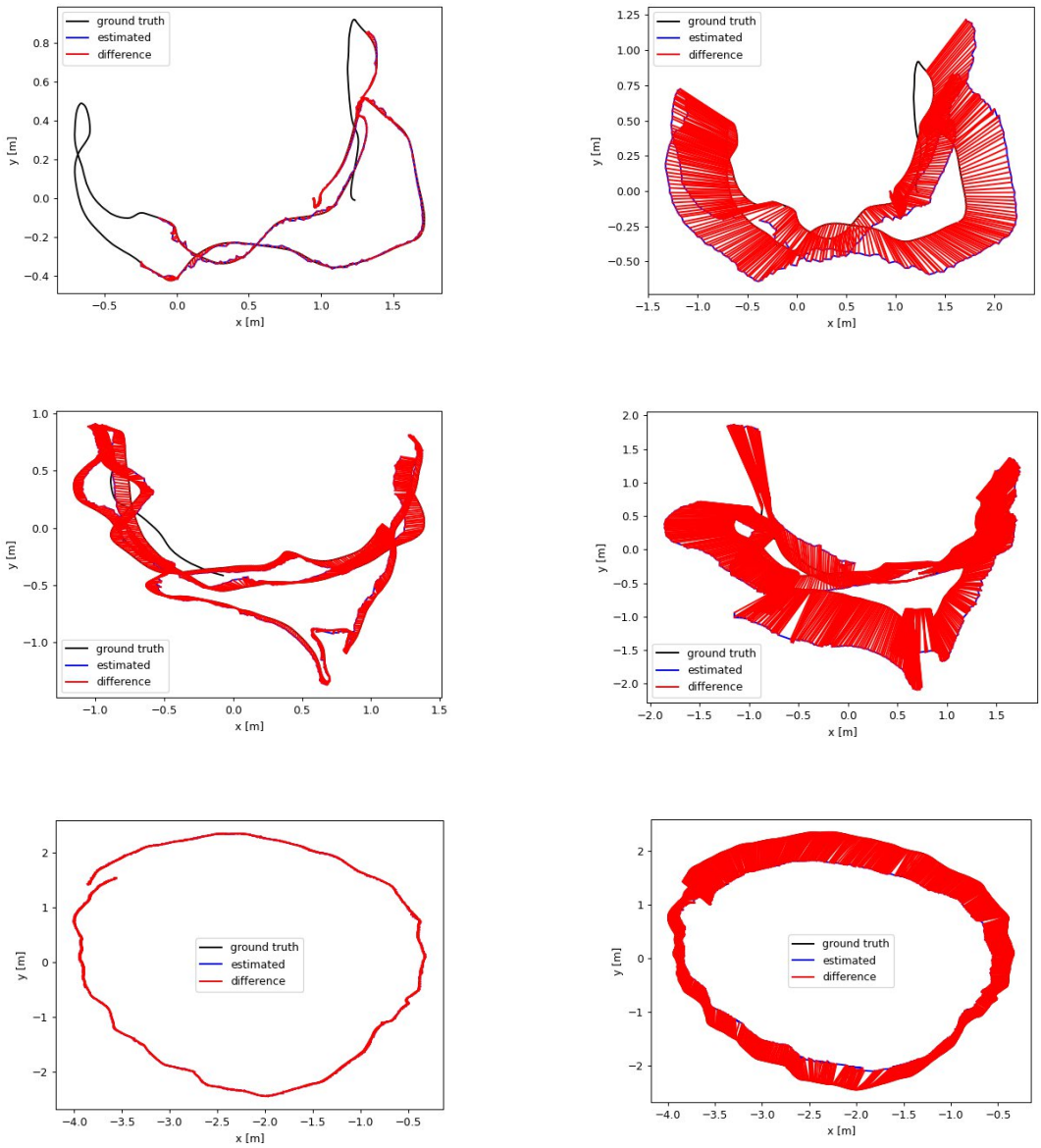


*Figure 3.1: Trajectories of 3 sequences*

*fr1_desk, fr1_room, Fr3_no_structure_texture_near_with_loop from top to bottom respectively, (the more the red color meaning the different between ground truth and estimate is big).*

### 3.3.2 KITTI dataset:

*Table 3: ORB-SLAM performance under KITTI dataset sequences.*

| Depth estimate method / Sequences | Depth-Anything-V2 | | stereo camera | | Duration (seconds) |
|---|---|---|---|---|---|
| | ATE (m) | RPE (m) | ATE (m) | RPE (m) | |
| 00 | 26.71 | 0.91 | 1.3 | 0.7 | 470 s |
| 01 | 462.34 | 6.76 | 10.4 | 1.39 | 114 s |
| 03 | 18.7 | 1.12 | 0.6 | 0.71 | 82 s |

| | | | | | |
|---|---|---|---|---|---|
| 05 | 24,98 | 1.00 | 0.8 | 0.4 | 287 s |

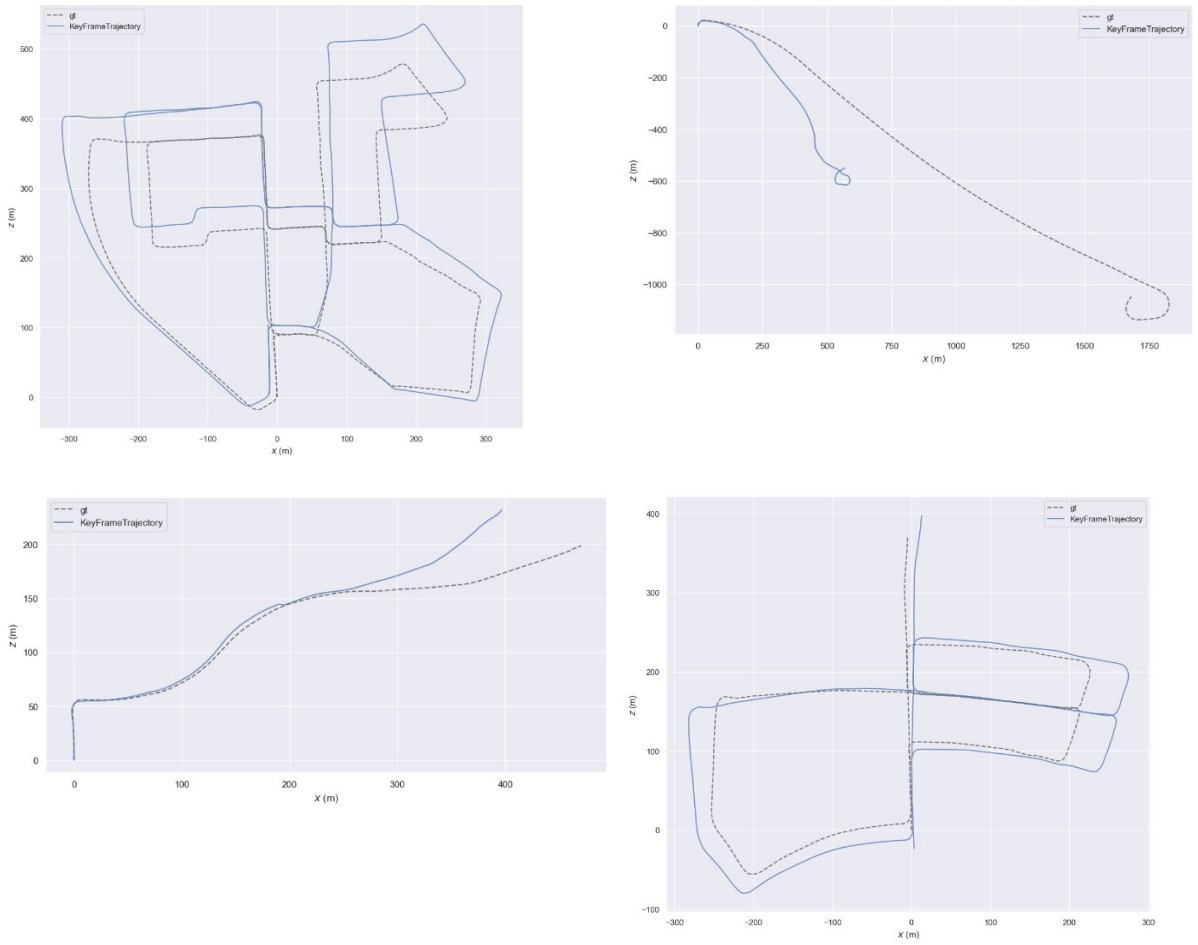


*Figure 3.2: Trajectory of 4 sequences*

*00 (top left), 01 (top right), 03 (bottom left), 05 (bottom right). (The strip line is ground truth, blue is the estimate).*

## 3.4 Conclusion:

Base on evaluation result show that the current performance of depth estimation model are not good. Moreover, the system can not handle outdoor scene with dynamic objects frequently appear in the scene resulting in hugh error rate when comparing with sensor.

One major drawback of Depth-Anything-V2 was that it only handle current frame independently and doesn't consider previous frames. Upon inspecting its generated depth images, we observed *flickering phenomenon* for a consecutive frames as shown in Figure 3.3. Consequently, object depth between frames is not consistent and in turn the performance to deteriorate over time. To our knowledge, there is another version of Depth-Anything-V2 called Video-Depth-Anything [10] that can handle frame sequentially, but due to constraint in time, we have to leave it for future improvement.

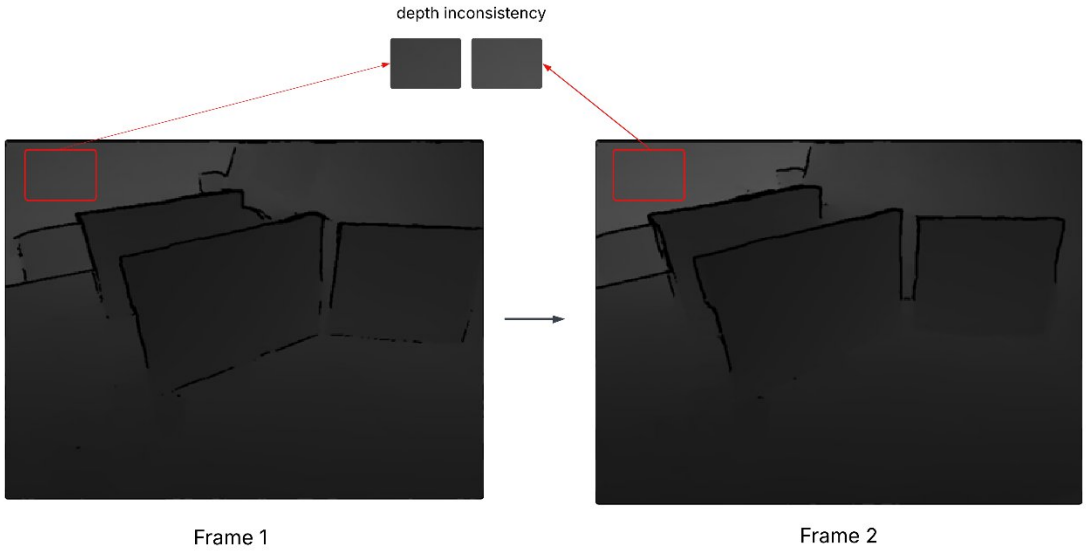


*Figure 3.3: Depth flickering or inconsistency between frames*

Despite of the drawback, we can still obtain a reasonable visual of the camera trajectory, this open another methods to tackle SLAM problem using deep learning to replace hardware.

# CHAPTER 4: APPENDIX

## 4.1 Covisibility graph and Essential graph:

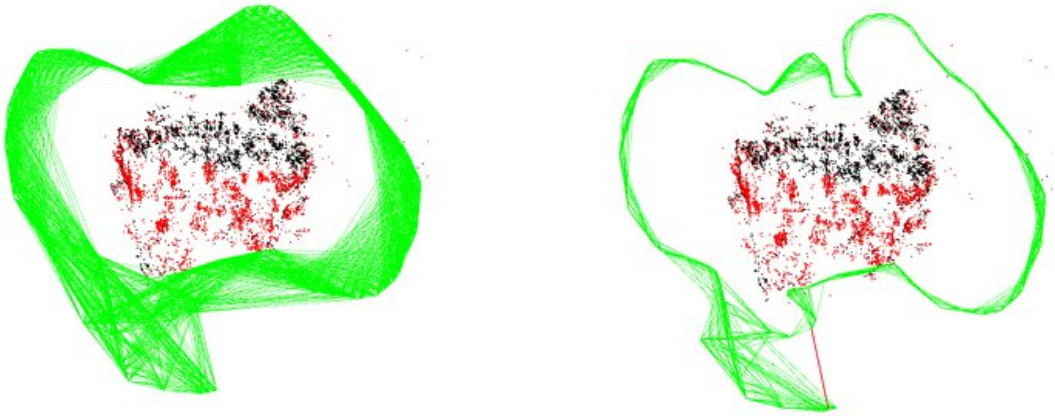

*Figure 4.1: Covisibility graph (left), Essential graph (right). [4]*

Covisibility information between keyframes is a very useful structure in the system, and is represented as an undirected weighted graph. Each node is a keyframe and an edge between two keyframes exists if they share observations of the same map points (at least 15), being the weight of the edge the number of common map points.

In order to correct a loop we perform a pose graph optimization that distributes the loop closing error along the graph. In order not to include all the edges provided by the covisibility graph, which can be very dense, we propose to build an Essential Graph that retains all the nodes (keyframes), but less edges, still preserving a strong network that yields accurate results.

The system builds incrementally a spanning tree from the initial keyframe, which provides a connected subgraph of the covisibility graph with minimal number of edges. When a new keyframe is inserted, it is included in the tree linked to the keyframe which shares most point observations.

## 4.2 Bags of Words Place Recognition:

The system has embedded a bags of words place recognition module, based on DBoW2[6], to perform loop detection and relocalization.

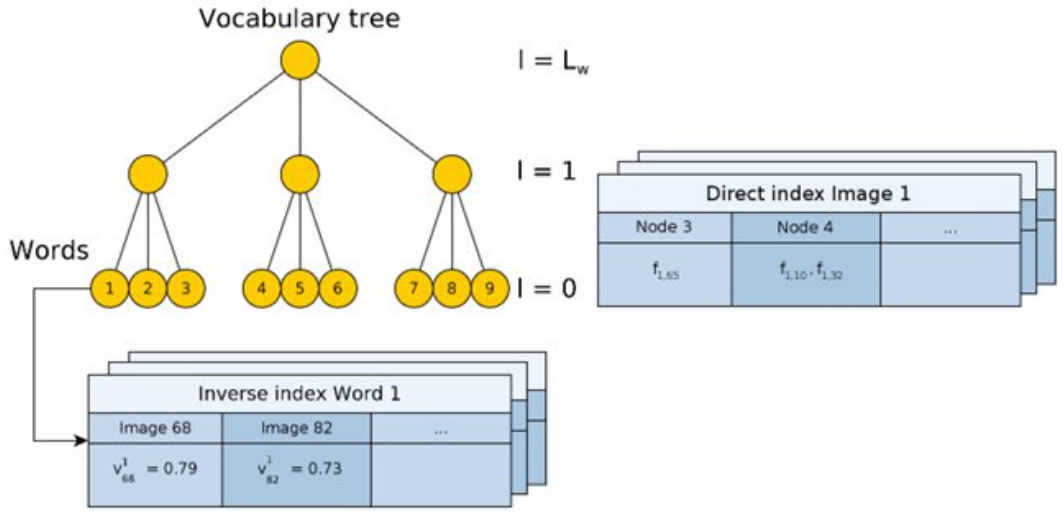


*Figure 4.2: Example of vocabulary tree*

*and direct and inverse indexes that compose the image database. The vocabulary words are the leaf nodes of the tree. The inverse index stores the weight of the words in the images in which they appear. The direct index stores the features of the images and their associated nodes at a certain level of the vocabulary tree. [6]*

In order to detect revisited places the system uses an image database composed of a hierarchical bag of words and direct and inverse indexes. The bag of words is a technique that uses a visual vocabulary to convert an image into a sparse numerical vector, allowing to manage big sets of images. The visual vocabulary is created offline by discretizing the descriptor space into $W$ visual words and structured as a tree.

To convert an image $I_t$, taken at time $t$, into a bag-of-words vector $v_t \in \mathbb{R}^W$, the binary descriptors of its features traverse the tree from the root to the leaves, by selecting at each level the intermediate nodes that minimize the Hamming distance.

To measure the similarity between two bag-of-words vectors $v_1$ and $v_2$, we calculate a $L1$-score $s(v_1, v_2)$, whose value lies in [0..1]:

$$s(v_1, v_2) = 1 - \frac{1}{2}\left|\frac{v_1}{|v_1|} - \frac{v_2}{|v_2|}\right|$$

Along with the bag of words, an inverse index is maintained. This structure stores for each word $w_i$ in the vocabulary a list of images $I_t$. It allows to perform comparisons only against those images that have some word in common with the query image. These two structures (the bag of words and the inverse index) are often the only ones used in the bag-of-words approach for searching images.

### 4.3 PnP (Perspective-n-Point):

PnP (Perspective-n-Point) is a method to solve 3D to 2D motion estimation. It describes how to estimate the camera's pose when the n 3D space points and their projection positions 2D are known. Here is the procedure for RANSAC base PnP for finding camera pose:

1) Randomly samples a **minimal set** of correspondences (typically 4-6 points for PnP). Each correspondence consists of:
2) **3D point in world coordinates**: a point with known 3D position $(x, y, z)$ in the world/map frame
3) **2D point in image coordinates**: the corresponding pixel location $(u, v)$ where that 3D point is observed in the camera image
4) Given these correspondences, PnP solves for the **camera pose** (position and orientation) that would project the 3D points to their observed 2D locations.
5) Computes camera pose (rotation $R$ and translation $t$) from the **minimal set**. This is the hypothesis/model generated from the sample
6) Tests all correspondences against the computed pose. Counts how many points agree with this model (within some threshold).
7) Keeps track of the model with the **maximum consensus** (most inliers). The model with most support is likely correct.
8) When a good model is found, refines it using **all inliers** (not just the minimal set). If refinement succeeds, returns immediately (early termination).
9) If maximum iterations reached or Requested number of iterations completed.

# REFERENCES


[1] *A comprehensive overview of Fish-Eye camera distortion correction methods*. (n.d.). https://arxiv.org/html/2401.00442v2

[2] Mao, L. (2022, August 21). Camera Intrinsics and Extrinsics. *Lei Mao's Log Book*. https://leimao.github.io/blog/Camera-Intrinsics-Extrinsics/

[3] Feng, J., Huang, Z., Kang, B., Xu, X., Yang, L., Zhao, H., & Zhao, Z. (2024). Depth Anything V2. *arXiv:2406.09414*, 21875–21911. https://doi.org/10.52202/079017-0688

[4] Mur-Artal, R., Montiel, J. M. M., Tardos, J. D., Mur-Artal, R., Montiel, J. M. M., & Tardos, J. D. (2015). ORB-SLAM: a versatile and accurate monocular SLAM system. *IEEE Transactions on Robotics*, *31*(5), 1147–1163. https://doi.org/10.1109/tro.2015.2463671

[5] Mur-Artal, R., Tardos, J. D., Mur-Artal, R., & Tardos, J. D. (2017). ORB-SLAM2: an Open-Source SLAM system for monocular, stereo, and RGB-D cameras. *IEEE Transactions on Robotics*, *33*(5), 1255–1262. https://doi.org/10.1109/tro.2017.2705103

[6] Galvez-López, D., & Tardos, J. (2012). Bags of Binary Words for Fast Place Recognition in Image Sequences. *IEEE Transactions on Robotics, 28(5), 1188-1197.* https://doi.org/10.1109/TRO.2012.2197158

[7] Xiang Gao, Tao Zhang, Yi Liu, & Qinrui Yan (2017). *14 Lectures on Visual SLAM: From Theory to Practice*. Publishing House of Electronics Industry.

[8] J. Sturm, N. Engelhard, F. Endres, W. Burgard, and D. Cremers, "A benchmark for the evaluation of RGB-D SLAM systems," in IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), Vilamoura, Portugal, October 2012, pp. 573–580.

[9] A. Geiger, P. Lenz, C. Stiller, and R. Urtasun, "Vision meets robotics: The KITTI dataset," The International Journal of Robotics Research, vol. 32, no. 11, pp. 1231–1237, 2013.

[10] Chen, S., Guo, H., Zhu, S., Zhang, F., Huang, Z., Feng, J., & Kang, B. (2025). Video depth anything: Consistent depth estimation for super-long videos. In *Proceedings of the Computer Vision and Pattern Recognition Conference* (pp. 22831-22840).